



FortuneCloud

FortuneCloud Technologies Group

2nd Floor, Abhinav Apartment, Beside Congress Bhavan, Shivaji
Nagar, Pune - 411005

Landmark: Near Pune Municipal Corporation (Ma.na.pa) Bus Stand, Pune

Contact No: 9766439090 / 7083777567

Website: www.fortunecloudindia.com

What is Java?



- ❑ Java is a popular programming language, created in 1995.
- ❑ It is owned by Oracle, and more than **3 billion** devices run Java
- ❑ It is open-source and free.
- ❑ It has a huge community support (tens of millions of developers)
- ❑ **Write Once, Run Anywhere.**
- ❑ Java runs on a variety of platforms, such as Windows, Mac OS, and the various versions of UNIX.

Java Version History

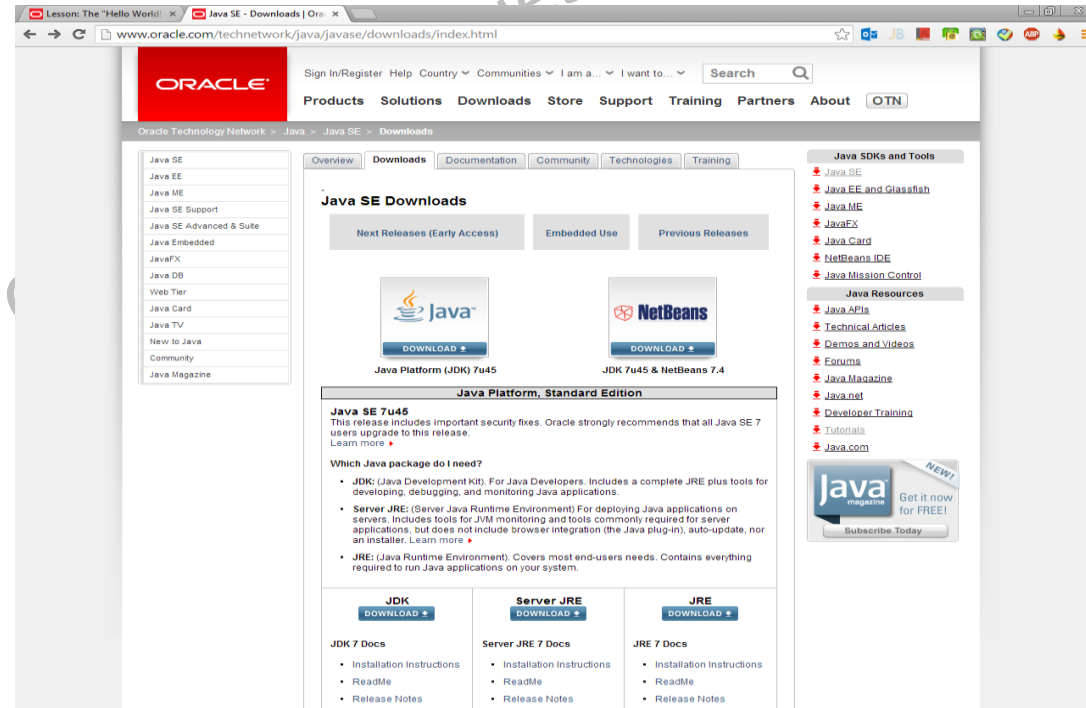
- ❑ JDK Alpha and Beta (1995)
- ❑ JDK 1.0 (23rd Jan 1996)
- ❑ JDK 1.1 (19th Feb 1997)
- ❑ J2SE 1.2 (8th Dec 1998)
- ❑ J2SE 1.3 (8th May 2000)
- ❑ J2SE 1.4 (6th Feb 2002)
- ❑ J2SE 5.0 (30th Sep 2004)
- ❑ Java SE 6 (11th Dec 2006)
- ❑ Java SE 7 (28th July 2011)
- ❑ Java SE 8 (18th Mar 2014)
- ❑ Java SE 9 (21st Sep 2017)
- ❑ Java SE 10 (20th Mar 2018)
- ❑ Java SE 11/12/13/14(2020)

Features of Java

- ☐ Simple
- ☐ Object-Oriented
- ☐ Platform independent
- ☐ Secured
- ☐ Interpreted
- ☐ High Performance
- ☐ Multithreaded

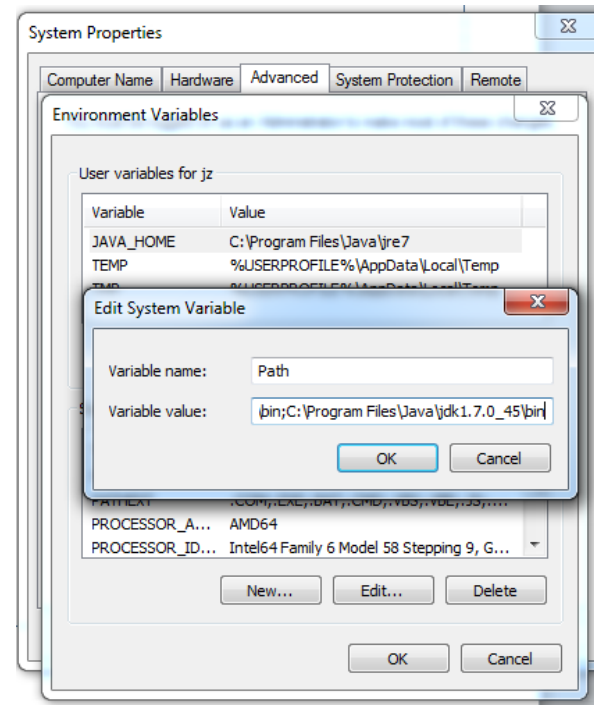
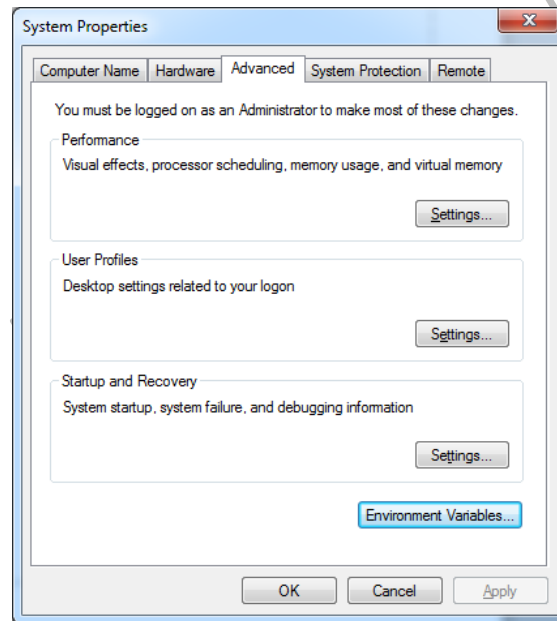
Installing Java in your machine

- Downloading Java Development Kit (JDK) from [Oracle](#)
- Java Runtime Environment (JRE) is usually included in the JDK installation file.



Installing Java in your machine

- Setting JAVA_HOME (Windows):
 - E.g., *C:\Program Files\Java\jdk1.8.0_45*
- Setting **path** and **classpath**

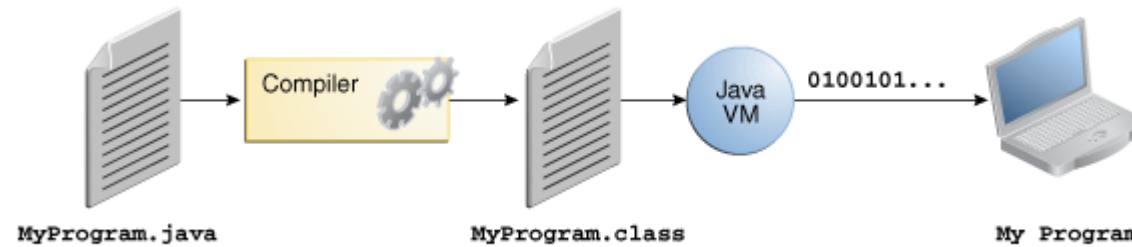


First java program

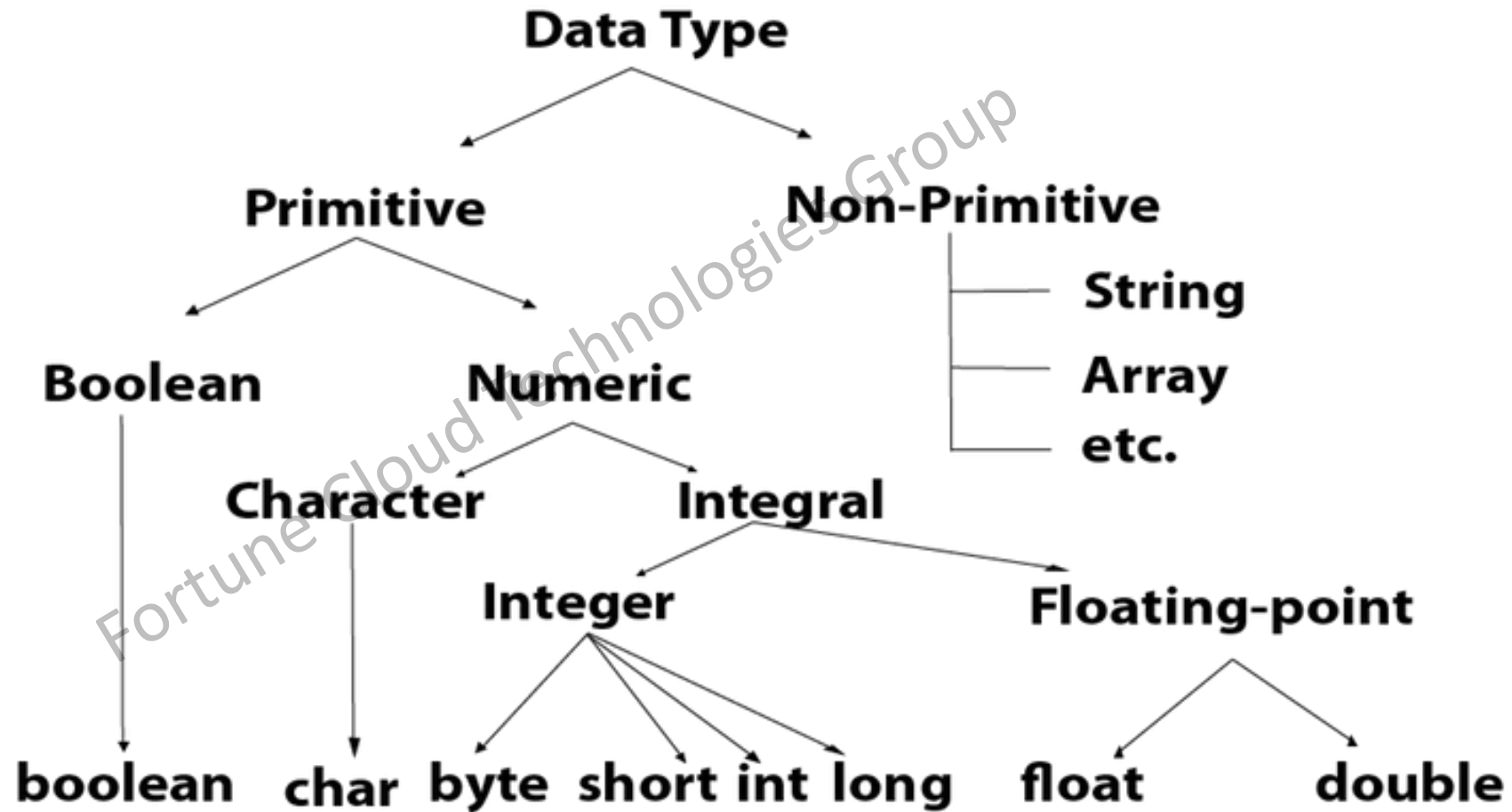
```
class Demo
{
    public static void main(String args[])
    {
        System.out.println("Hello World");
    }
}
```

Java Development Process

.java => .class => JVM execution



Data Types



Variables

- ❑ A variable is a container which holds the value while the program is executed.
- ❑ A variable is assigned with a data type.

data type variable Name = initial Value;

Ex. int data=20;

Types Variables



Type of Variables

1) Local Variable

inside the body of
the method

2) Instance Variable

inside the class but
outside the body of
the method

3) Static variable

declared as static

Example

```
class Demo
{
    int data=50;//instance variable

    static int Val=100;//static variable

    void method()
    {
        int count=90;//local variable
    }
}
```

Operators

❑ **Operator** is a symbol which is used to perform operations.

Unary	postfix	<i>expr++ expr--</i>
	prefix	<i>++expr --expr</i>
Arithmetic	multiplicative	<i>* / %</i>
	additive	<i>+ -</i>
Relational	comparison	<i>< > <= >=</i>
	equality	<i>== !=</i>
Logical	logical AND	<i>&&</i>
	logical OR	<i> </i>
Ternary	ternary	<i>? :</i>

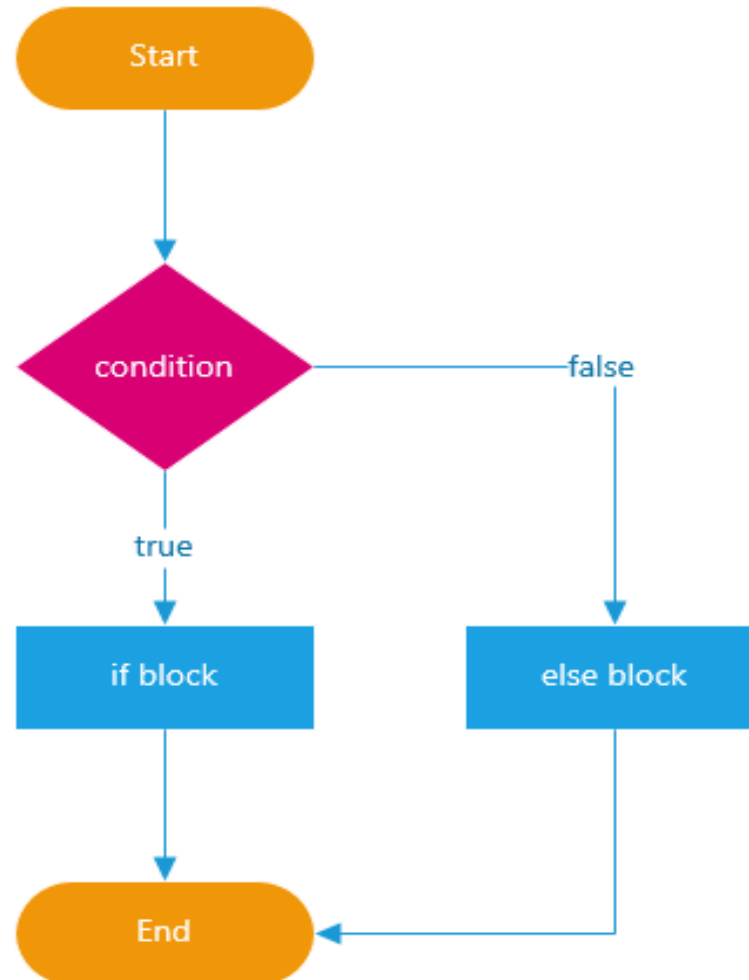
Keywords

boolean	byte	char	double	float
short	void	int	long	while
for	do	switch	break	continue
case	default	if	else	try
catch	finally	class	abstract	extends
final	import	new	instance of	private
interface	native	public	package	implements
protected	return	static	super	synchronized
this	throw	throws	transient	volatile

If-else Statement

- ❑ if statement
- ❑ if-else statement
- ❑ if-else-if ladder
- ❑ nested if statement

Flow diagram



if(condition) {
 // statement
} else {
 // statement
}

❏ Example

Ex. Even odd number

```
int num=10;

if(num%2==0)
{
    System.out.println("Even Number");
}
else
{
    System.out.println("Odd Number");
}
```

if-else-if

```
if(condition1)
{
    //code to be executed if condition1 is true
}
else if(condition2)
{
    //code to be executed if condition2 is true
}
else if(condition3)
{
    //code to be executed if condition3 is true
}
else
{
    //code to be executed if all the conditions are false
}
```

Switch

```
switch(expression)
{
    case value1:
        //code to be executed;
        break;

    case value2:
        //code to be executed;
        break;

    .....

    default:
        code to be executed if all cases are not matched;
}
```

Example

```
int number=40;

switch(number)
{
    case 10:
        System.out.println("10");
        break;
    case 20:
        System.out.println("20");
        break;
    case 30:
        System.out.println("30");
        break;
    default:
        System.out.println("Not match");
}
```

For loop

```
for(initialization ; condition ; inc/dec)
{
    //code to be executed
}
```

While

```
while(condition)
{
    //code to be executed
}
```

```
int i=1;
while(i<=10){
    System.out.println(i);
    i++;
}
```

do-while

Syntax:-

```
do  
{  
    //code to be executed  
}while(condition);
```

Example:-

```
int i=1;  
    do{  
        System.out.println(i);  
        i++;  
    }while(i<=10);
```


Fortune Cloud Technologies Group



- fixed-size sequential collection of elements of the same type.
- collection of similar type of data items stored at contiguous memory locations.
- store the primitive type of data such as int, char, double, float, etc.
- each data element can be randomly accessed by using its index number.
- The lowest address corresponds to the first element and the highest address to the last element.

Advantages

Code Optimization

Ease of traversing

Ease of sorting

Random Access

Types of Array

One dimensional array

Two dimensional array

Multi-dimensional array

One dimensional array

```
import java.util.Scanner;

public class sample
{
    public static void main(String[] args)
    {
        int arr[]=new int[5];

        Scanner sc=new Scanner(System.in);

        System.out.println("Enter the array elements");

        for(int i=0;i<arr.length;i++)
        {
            arr[i]=sc.nextInt();
        }
    }
}
```

```
System.out.println("Array elements are:");
for(int i=0;i<arr.length;i++)
{
    System.out.println(arr[i]);
}
}
```

Two dimensional array

```
import java.util.Scanner;

public class sample
{
    public static void main(String[] args)
    {
        int arr[][]=new int[2][2];
        int i,j;

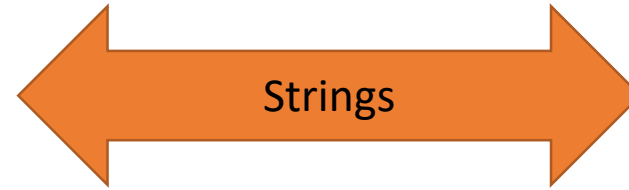
        Scanner sc=new Scanner(System.in);
        System.out.println("Enter the array elements");

        for(i=0;i<2;i++)
        {
            for(j=0;j<2;j++)
            {
                arr[i][j]=sc.nextInt();
            }
        }
    }
}
```

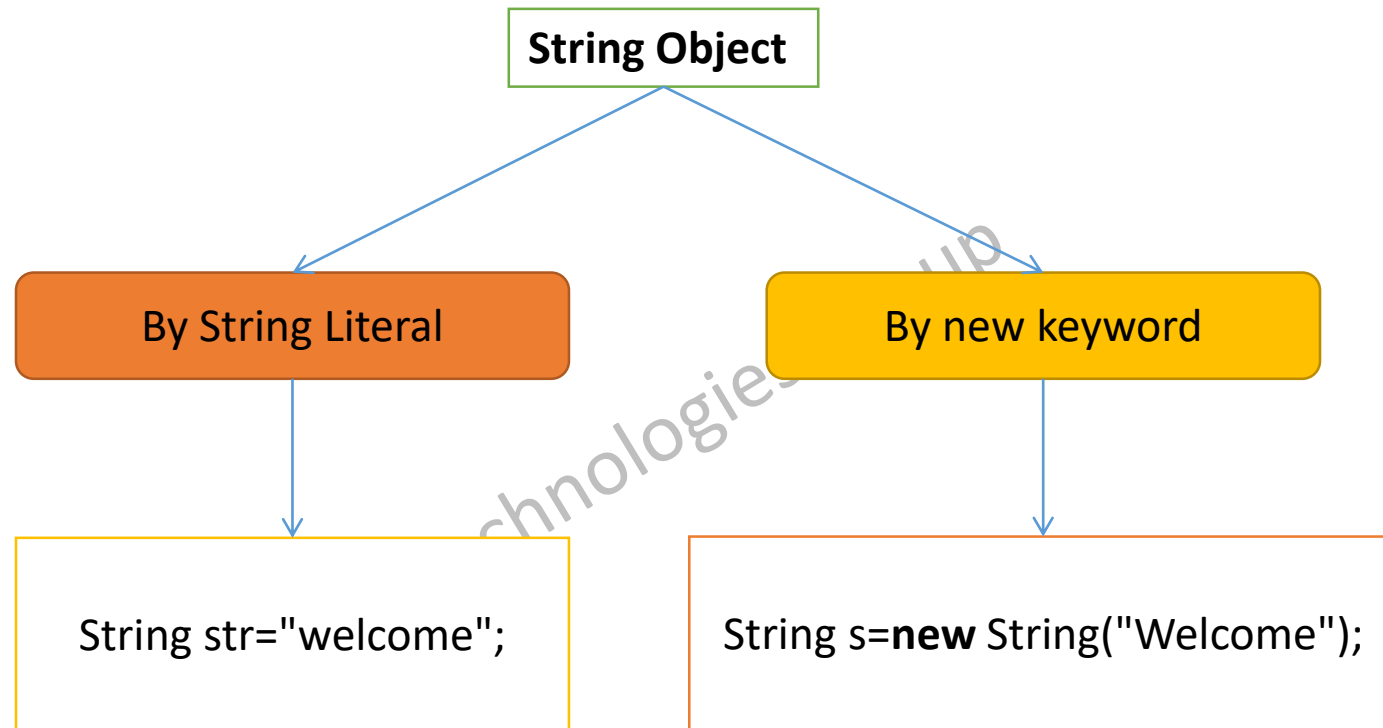
```
System.out.println("Array elements are:");
for(i=0;i<2;i++)
{
    for(j=0;j<2;j++)
    {
        System.out.print(arr[i][j]);
    }
}
}
```

Multi-dimensional array

```
int arr[][][] = new int[2][2][2];
```

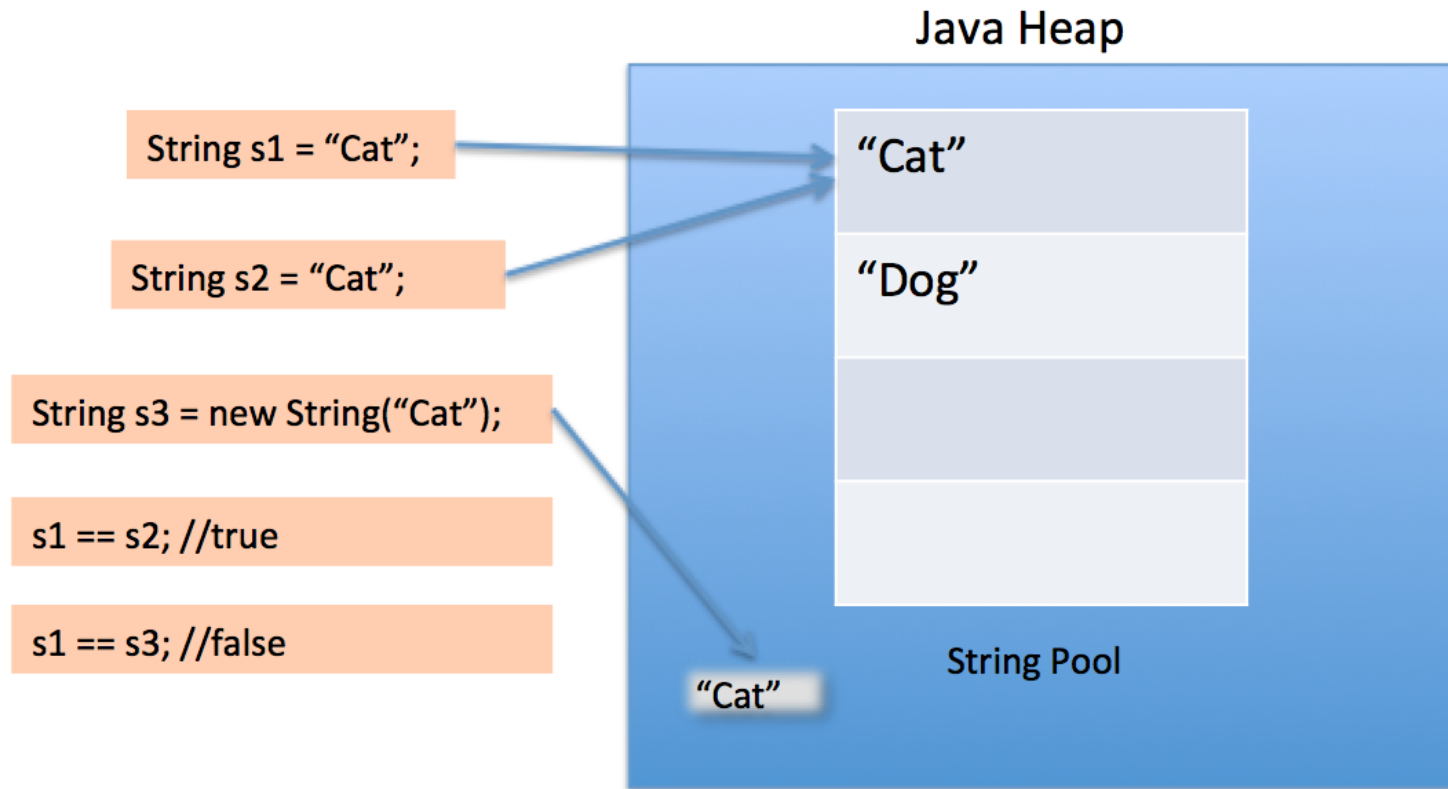


- One-dimensional array of characters terminated by a **null** character '\0'.
- The character array or the string is used to manipulate text such as word or sentences.



Java String literal is created by using double quotes.

Memory allocation in String



Example

```
public class String_example
{
    public static void main(String[] args)
    {
        String str1="hello";
        char ch[]= {'w','o','r','l','d'};

        System.out.println(str1);
        System.out.println(ch);
    }
}
```

charAt()

concat()

equals()

equalsIgnoreCase()

indexOf()

String Functions

length()

substring()

replace()

toUpperCase()

toLowerCase()

trim()

charAt()

```
String str1="hello";  
System.out.println(str1.charAt(0));
```

concat()

```
//immutable string  
str1.concat("world");  
System.out.println(str1);
```

```
//solution  
str1=str1.concat("world");  
System.out.println(str1);
```

equals()

```
String str1="hello";  
String str2="HELLO";
```

```
System.out.println(str1.equals(str2));
```

equalsIgnoreCase()

```
String str1="hello";  
String str2="HELLO";
```

```
System.out.println(str1.equalsIgnoreCase(str2));
```

indexOf()

```
String str1="hello";  
  
System.out.println(str1.indexOf('e'));
```

length()

```
String str1="hello";  
  
System.out.println(str1.length());
```

substring()

```
String str4="good afternoon";  
  
System.out.println(str4.substring(4));  
  
System.out.println(str4.substring(5,7));
```

trim()

```
String str3=" hello world ";  
  
System.out.println(str3.trim());
```

toLowerCase()

```
String str1="hello";  
str1.toLowerCase();
```

```
System.out.println(str1);
```

toUpperCase()

```
String str1="hello";  
str1.toUpperCase();
```

```
System.out.println(str1);
```

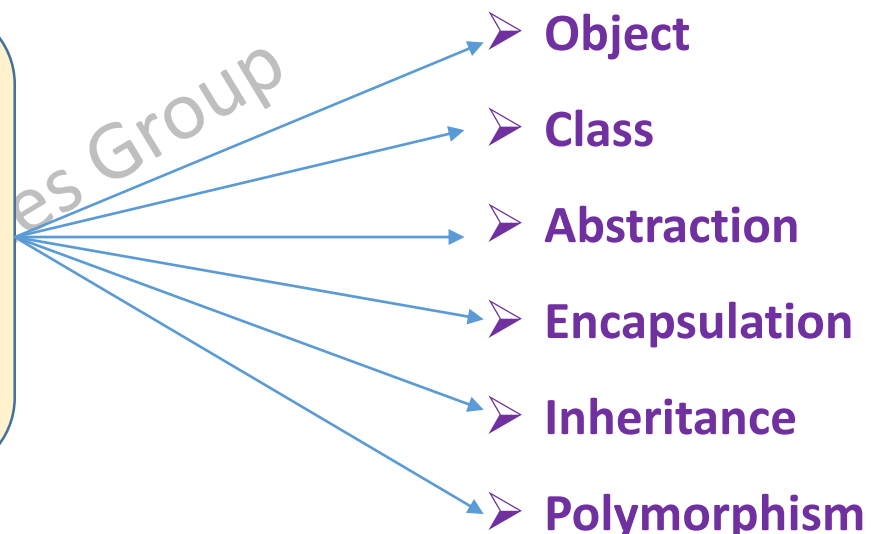
replace()

```
String str1="hello";  
System.out.println(str1.replace("ello", "ii"));
```

```
System.out.println(str1);
```

OOP (Object-Oriented Programming)

- The main aim of object-oriented programming is to implement real-world entities,
- for example, object, classes, abstraction, inheritance, polymorphism, etc.

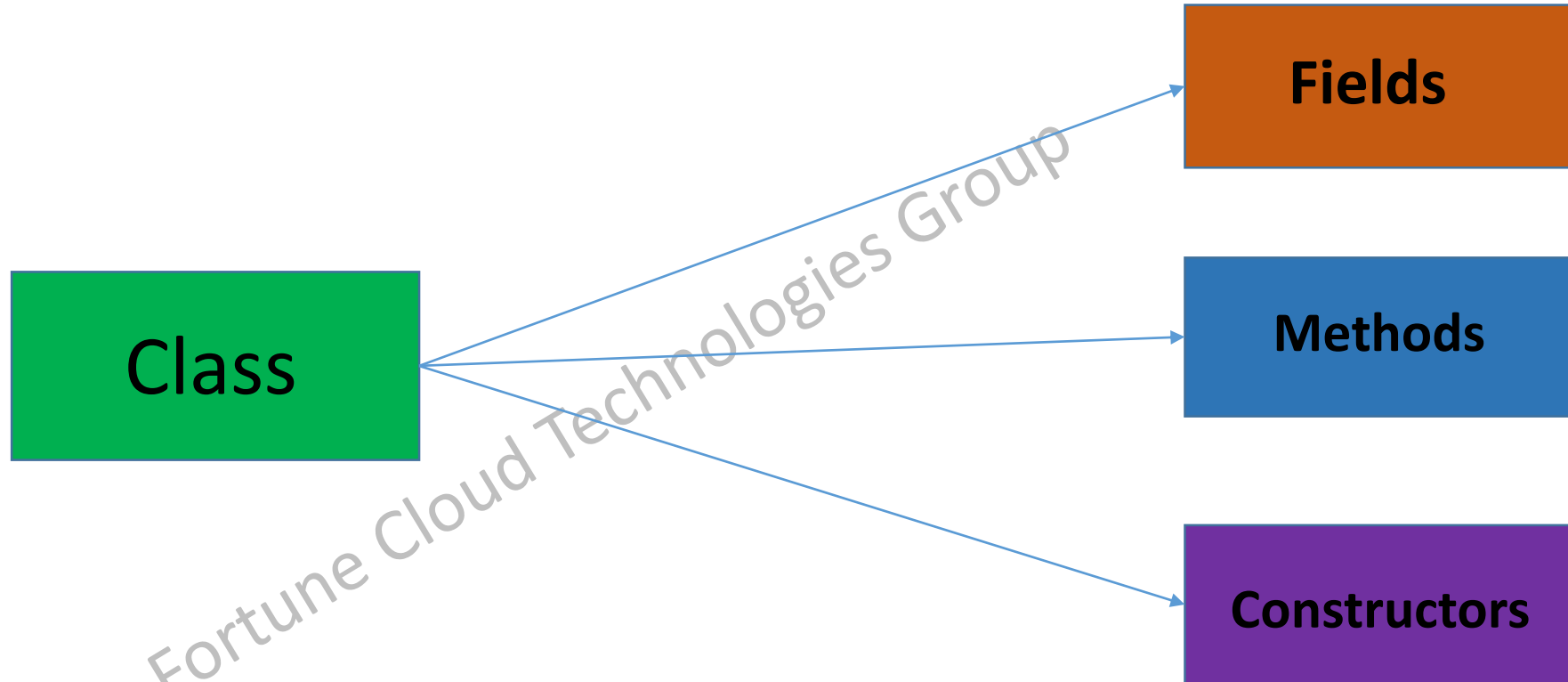
- 
- Object
 - Class
 - Abstraction
 - Encapsulation
 - Inheritance
 - Polymorphism

❑ Classes and Objects are basic concepts of Object Oriented Programming which revolve around the **real life entities**.

Class



- ❑ *Collection of objects* is called class.
- ❑ It is a logical entity.
- ❑ A class can also be defined as a blueprint from which you can create an individual object.
- ❑ Class doesn't consume any space.
- ❑ Ex. Student, Vehicle, Shape



Object

- ❑ In object-oriented programming technique, we design a program using **objects** and **classes**.
- ❑ An object in Java is the physical as well as a logical entity, whereas, a class in Java is a logical entity only.
- ❑ An entity that has **state**, **behaviour** and **identity** is known as an object.
- ❑ The object is *an instance of a class*.

Object

Characteristics of Object

A

State

Represents the data of an object.

Behavior

represents the behavior of an object such as deposit, withdraw, etc.

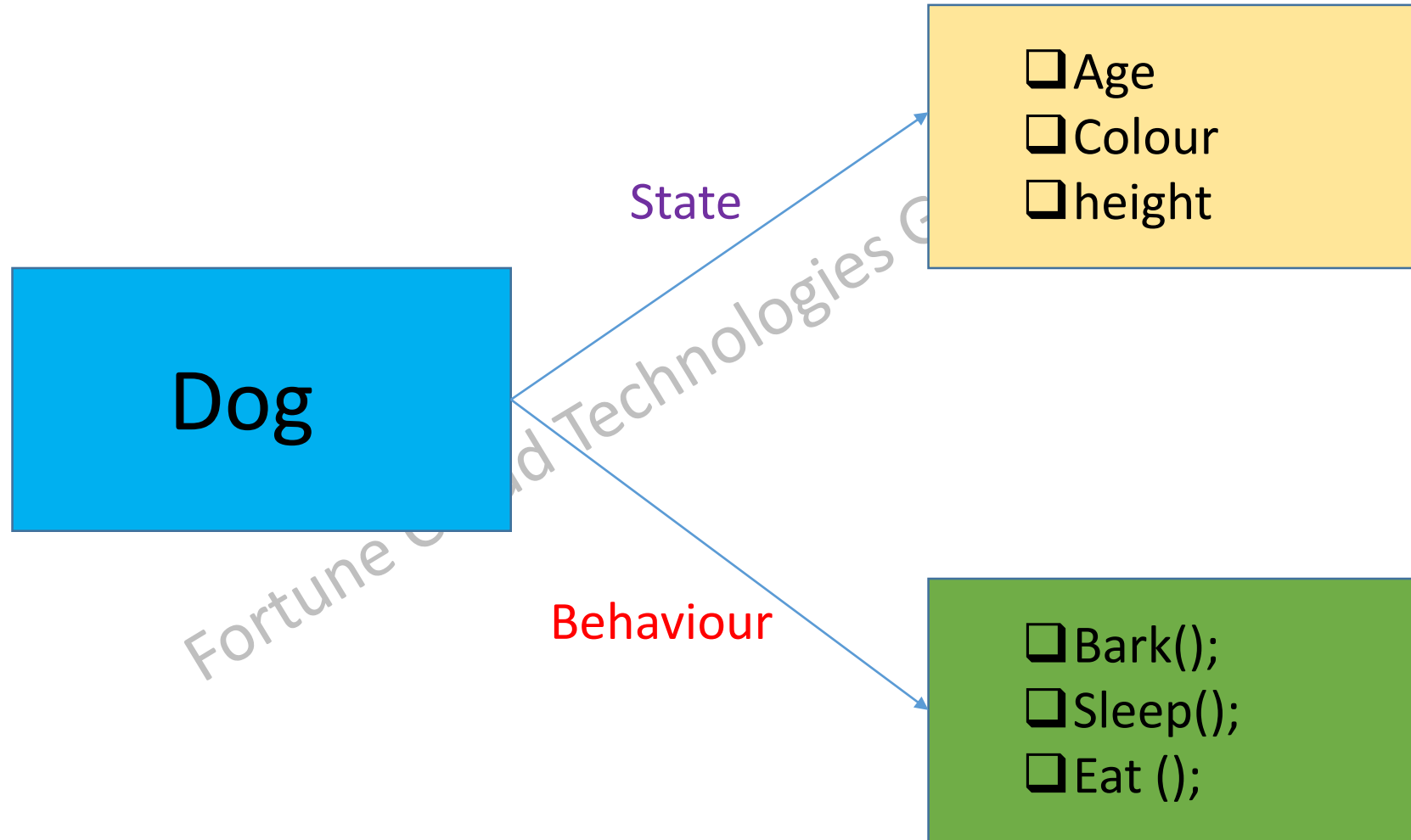
B

C

Identity

It is used internally by the JVM to identify each object uniquely.

Example



Syntax:-

```
class <class name>{  
    field;  
    method;  
}
```

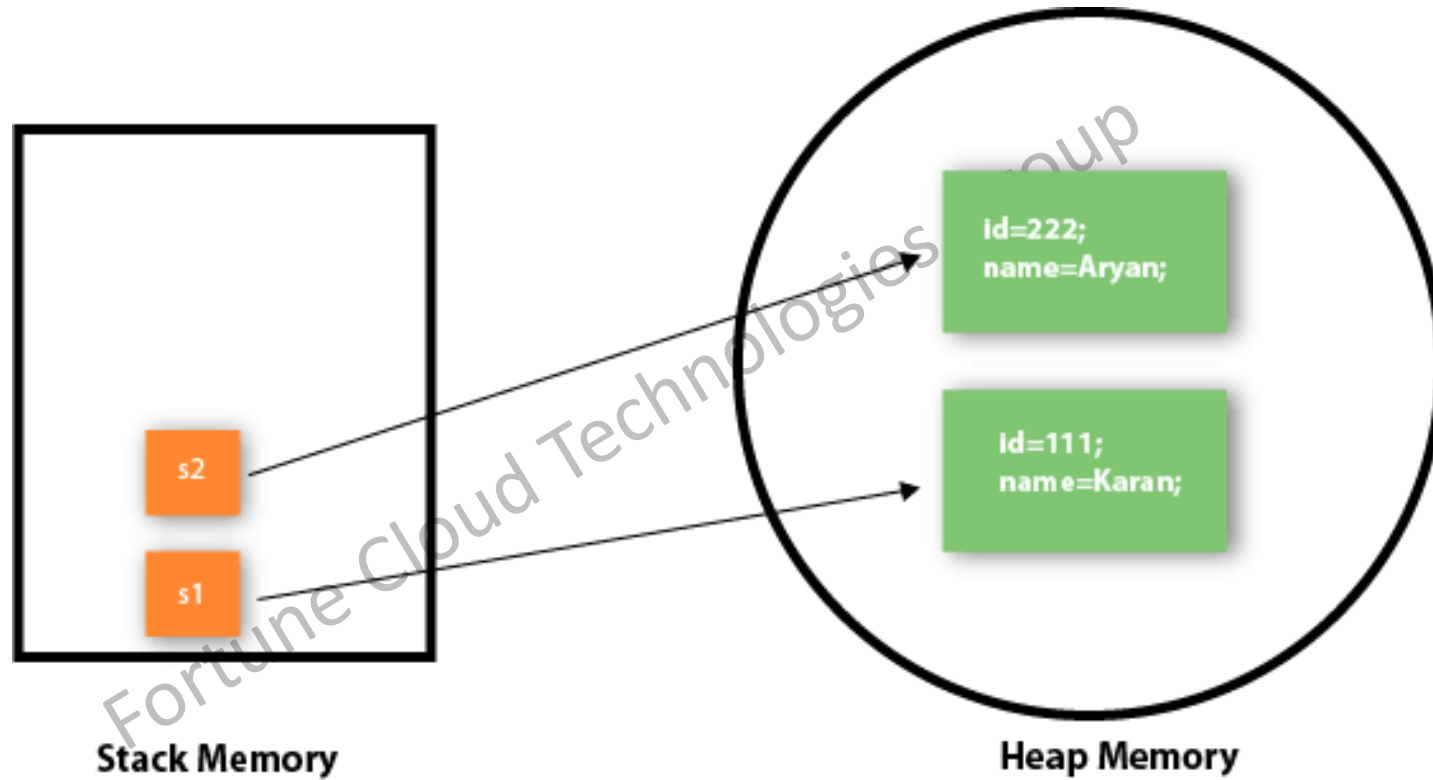
New keyword:-

- ❑ The new keyword is used to allocate memory at runtime.
- ❑ All objects get memory in Heap memory area.

Example

```
class Student{  
    int id;  
    String name;  
}  
class Test{  
    public static void main(String args[]){  
        Student s1=new Student();  
        s1.id=101;  
        s1.name="Karan";  
        System.out.println(s1.id+" "+s1.name);  
    }  
}
```

HEAP Memory



Constructor



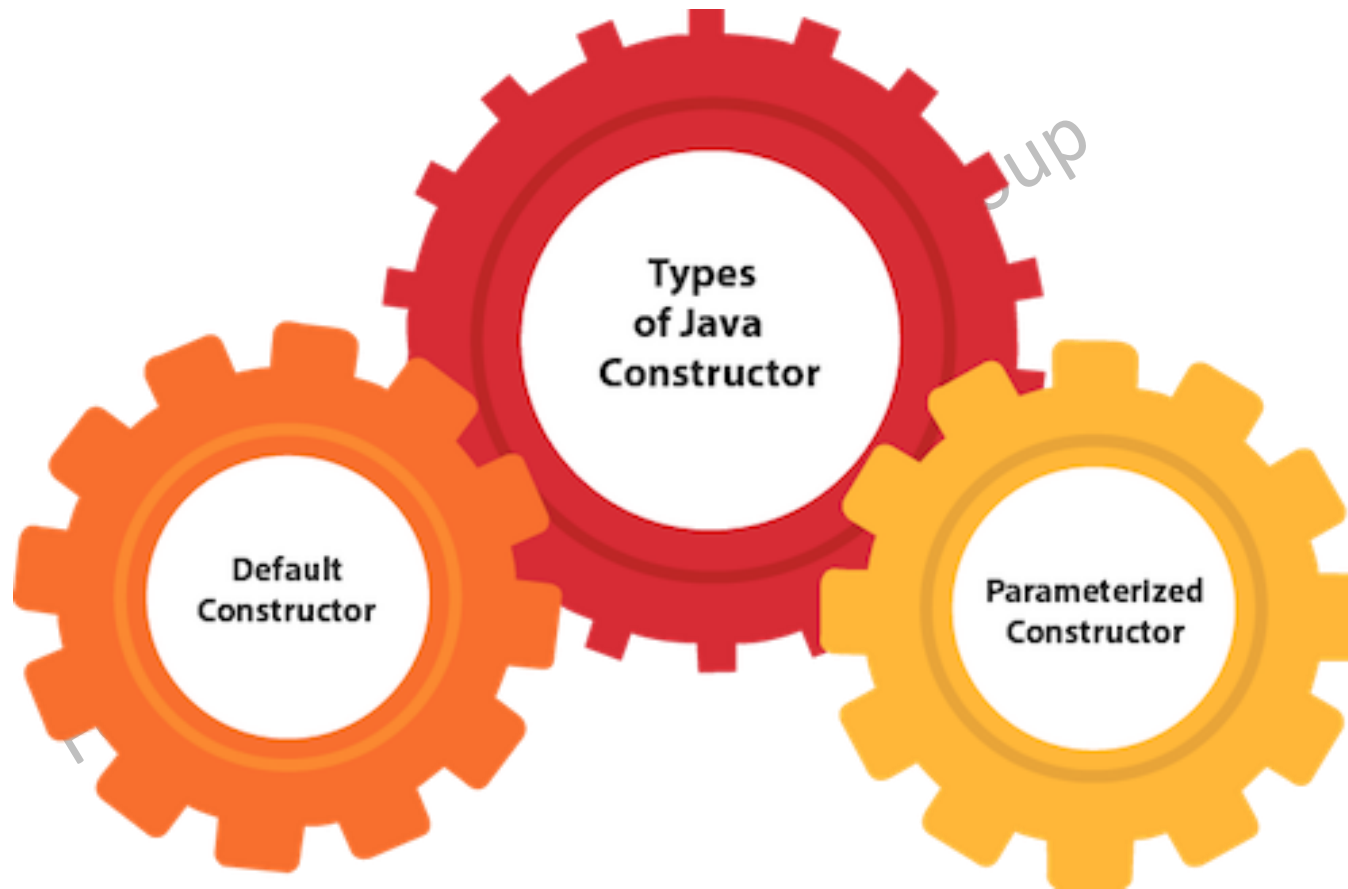
- ❑ a constructor is a block of codes similar to the method.
- ❑ It is called when an instance of the **class** is created.
- ❑ At the time of calling constructor, memory for the object is allocated in the memory.
- ❑ It is a **special type of method** which is used to initialize the object.
- ❑ Every time an object is created using the **new** keyword, **at least one constructor is called**.
- ❑ It calls a default constructor if there is no constructor available in the class. In such case, **Java compiler provides a default constructor by default**.

Rules for creating constructor



- ❑ **Constructor** name must be the same as its **class name**.
- ❑ A Constructor must have **no explicit return type**.
- ❑ A Java constructor cannot be **abstract, static, final, and synchronized**.
- ❑ we can have **private, protected, public or default** constructor in Java.

Types of Java constructors



Example

```
public static void main(String args[]){  
    //creating objects  
  
    Student s1=new Student(); //default  
  
    Student s2=new Student(101,"Tom");//parameterized
```

- The default constructor is used to provide the default values to the object like 0, null, etc., depending on the type.

```
class sample
{
    sample()
    {
        System.out.println("I am default constructor");
    }

    public static void main(String args[])
    {
        sample m1=new sample();
    }
}
```

Output



I am default constructor

Parameterized Constructor

- The parameterized constructor is used to provide different values to distinct objects.

```
class sample{  
    int id;  
    String name;  
  
    sample(int i, String n)  
    {  
        id = i;  
        name = n;  
    }  
    void show()  
    {  
        System.out.println("Id="+id);  
        System.out.println("Name="+name);  
    }  
}
```

```
public static void main(String args[])  
{  
    sample m1=new sample(101,"abc");  
    m1.show();  
}
```

Output

Id=101
Name=abc

- There is no copy constructor in Java. we can copy the values from one object to another.

```
class sample{  
    int id;  
    String name;  
  
    sample(int i, String n)  
    {  
        id = i;  
        name = n;  
    }  
  
    sample(sample s)  
    {  
        id = s.id;  
        name = s.name;  
    }  
}
```

```
void show()  
{  
    System.out.println("Id="+id);  
    System.out.println("Name="+name);  
}  
  
public static void main(String args[])  
{  
    sample m1=new sample(101,"abc");  
    m1.show();  
  
    sample m2=new sample(m1);  
    m2.show();  
}
```

Output



```
Id=101  
Name=abc  
Id=101  
Name=abc
```

Inheritance

- ❑ **Inheritance in Java** is a mechanism in which one object **acquires all the properties and behaviour's of a parent object.**
- ❑ The idea behind inheritance in Java is that you can create new classes that are built upon existing classes.
- ❑ When you inherit from an existing class, you can reuse methods and fields of the parent class. Moreover, you can add new methods and fields in your current class also.
- ❑ represents the **IS-A relationship** which is also known as a *parent-child* relationship

Why use inheritance in java

- ❑ For Code Reusability.

extends keyword

Syntax

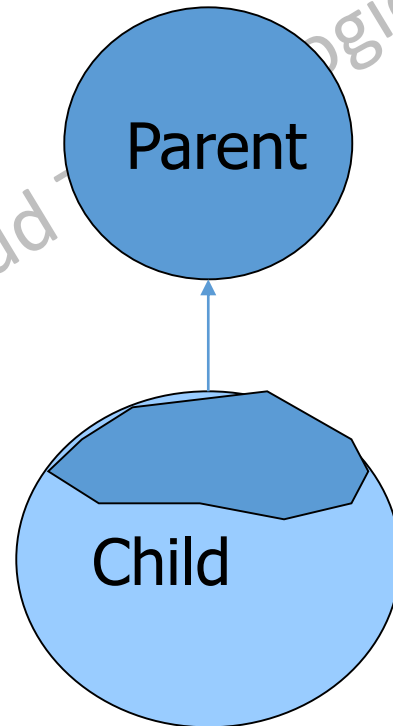
```
class Superclass_name{  
}
```

```
class Subclass-name extends Superclass-name  
{  
    //methods and fields  
}
```

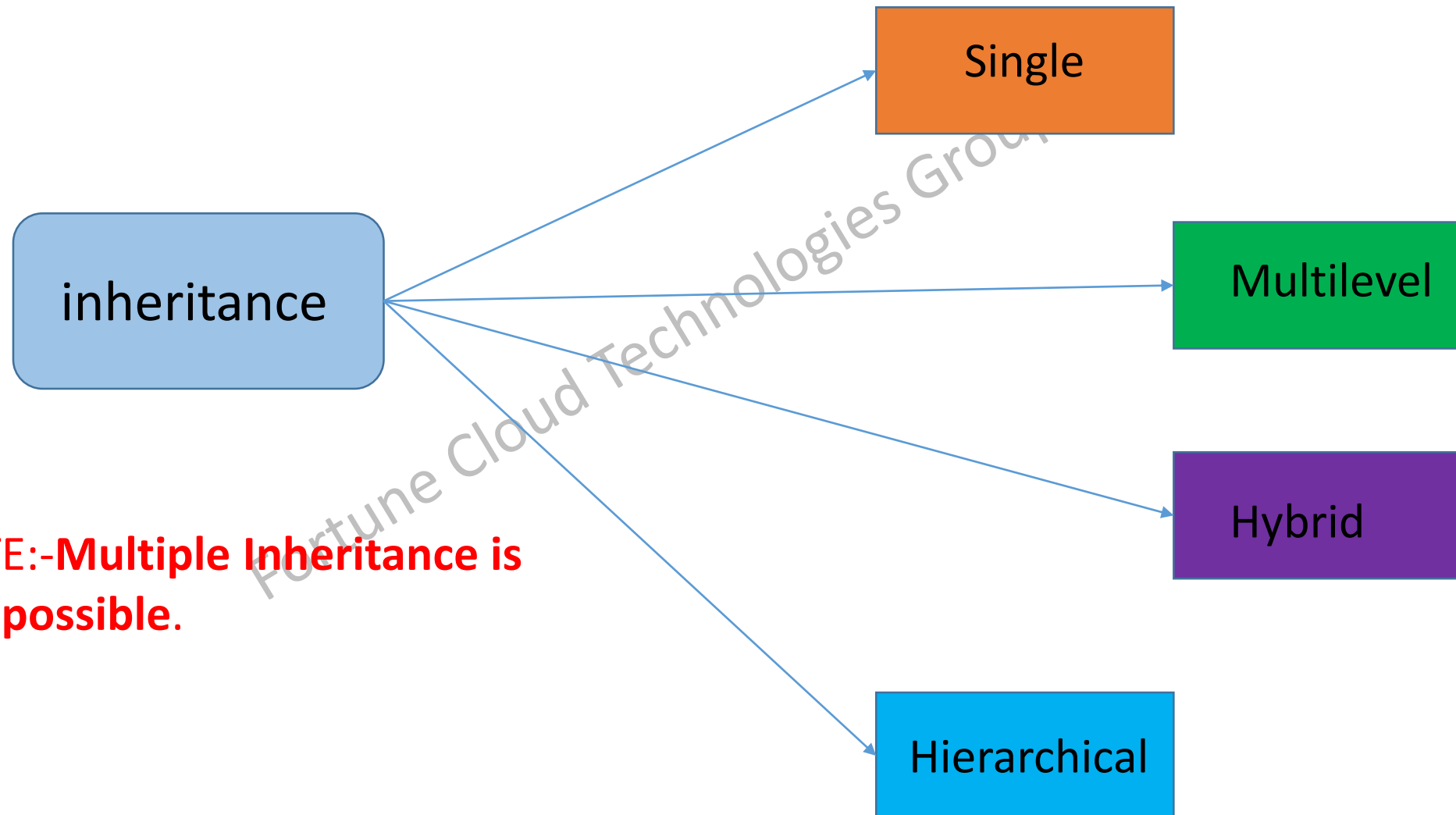
- ❑ new class that derives from an existing class.
- ❑ "extends" is to increase the functionality.

IS –A Relation

- Parent Class (Super Class) – Child Class (Sub Class)
- Subclass inherits properties from the parent class.

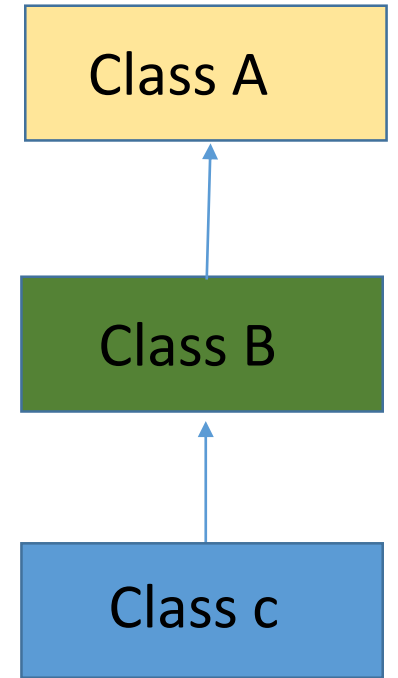
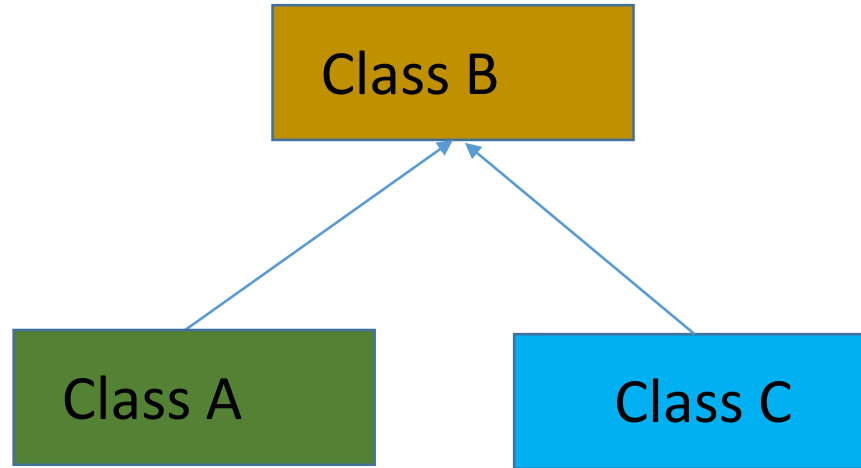
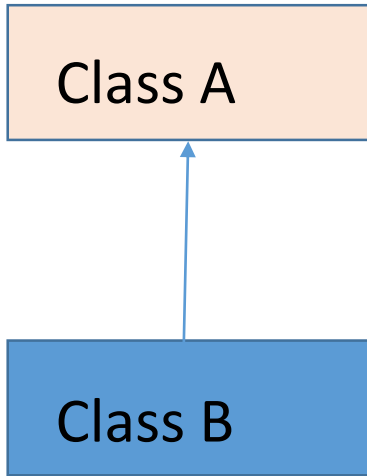


Types of inheritance

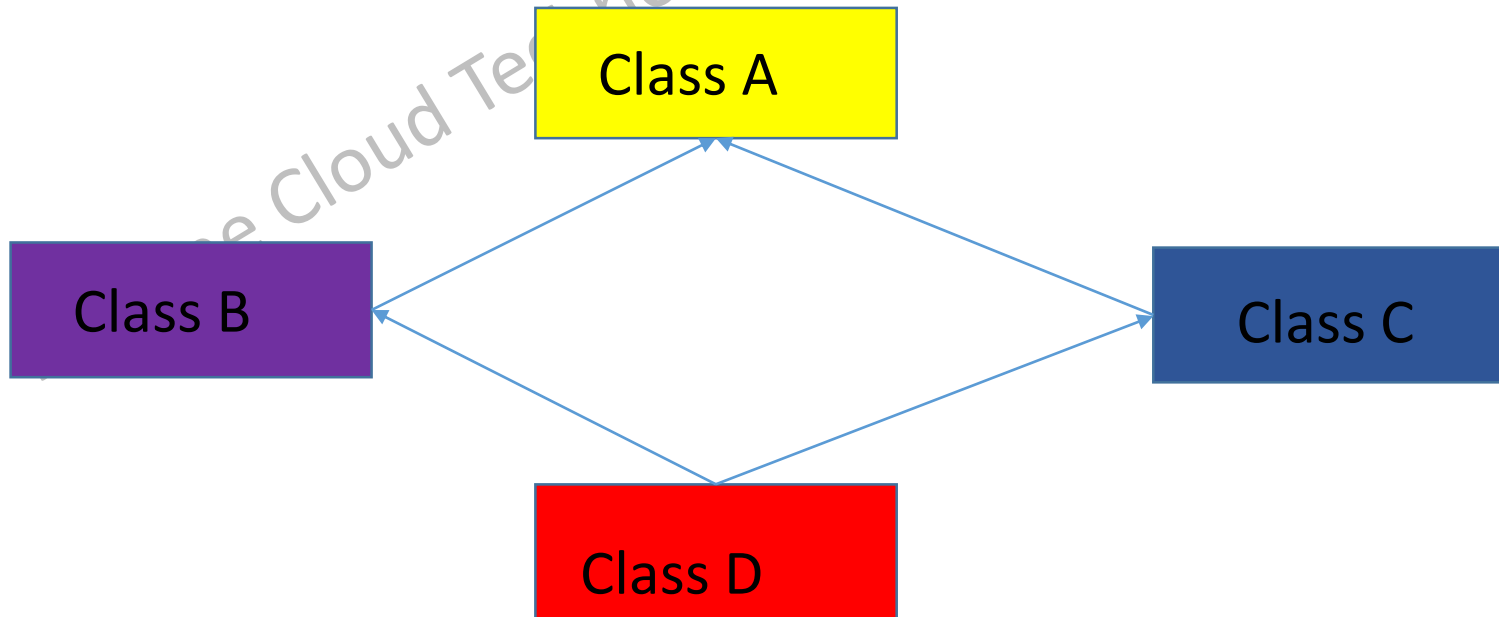


NOTE:-Multiple Inheritance is Not possible.

❖ Hierarchical



❖ Single



❖ Hybrid

❖ Multilevel

Interface

- The interface in Java is a mechanism to achieve abstraction
- There can be only abstract methods in the Java interface, not method body.
- Used to achieve abstraction and multiple inheritance
- Java Interface also **represents the IS-A relationship**.

Why use Interface

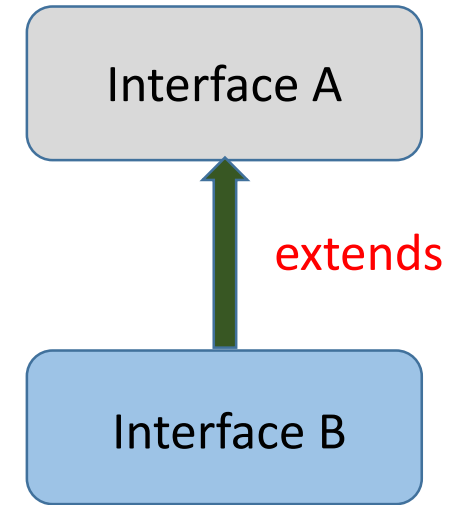
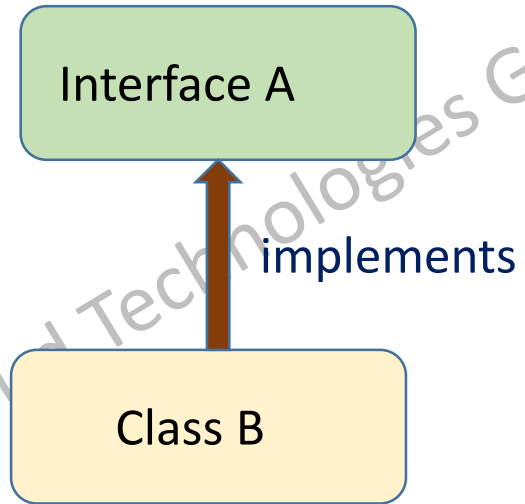
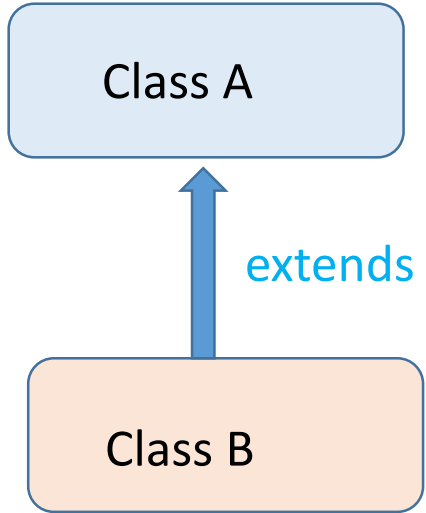
- It is used to achieve abstraction.
- By interface, we can support the functionality of multiple inheritance.

Example

```
interface test
{
    void payment();
}

class sample implements test
{
    public void payment()
    {
        System.out.println("Net banking");
    }

    public static void main(String args[])
    {
        sample s = new sample();
        s.payment();
    }
}
```



Access Modifiers

- specifies the accessibility or scope of a field, method, constructor, or class.
- We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.

Access Modifiers

	default	public	private	protected
Same Class	YES	YES	YES	YES
Same package subclass	YES	YES	NO	YES
Same package non-subclass	YES	YES	NO	YES
Different package subclass	NO	YES	NO	YES
Different package non-subclass	NO	YES	NO	NO



Private

- The private access modifier is accessible only within the class.

```
class Parent
{
    private int a=10;
    private void display()
    {
        System.out.println("Private method");
    }
}
```

```
public class sample extends Parent
{
    public static void main(String[] args)
    {
        sample s=new sample();
        s.a; //error
        s.display(); //error
    }
}
```



Public

- The **public access modifier** is accessible everywhere.

```
class Parent
{
    public void display()
    {
        System.out.println("Public method");
    }
}
```

```
public class sample extends Parent
{
    public static void main(String[] args)
    {
        sample s=new sample();
        s.display();
    }
}
```



Protected

- The **protected access modifier** is accessible within class, package and outside the class, package but through inheritance only.

```
class Parent
{
    protected void display()
    {
        System.out.println("Protected method");
    }
}
```

```
public class sample extends Parent
{
    public static void main(String[] args)
    {
        sample s=new sample();
        s.display();
    }
}
```



Default

- If you don't use any modifier, it is treated as **default** by default.

```
public class sample
{
    void display()
    {
        System.out.println("Default method");
    }

    public static void main(String[] args)
    {
        sample s=new sample();
        s.display();
    }
}
```

this keyword

- ❑ this can be used to refer current class instance variable.
- ❑ this() can be used to invoke current class constructor.
- ❑ this can be passed as an argument in the method call.
- ❑ this can be passed as argument in the constructor call.
- ❑ this can be used to return the current class instance from the method.

- The `this` keyword can be used to refer current class instance variable.

```
class sample{
    int id;
    String name;

    sample(int id, String name)
    {
        this.id = id;
        this.name = name;
    }

    void show()
    {
        System.out.println("Id="+id);
        System.out.println("Name="+name);
    }
}
```

```
public static void main(String args[])
{
    sample m1=new sample(101,"abc");
    m1.show();
}
```

Output

Id=101
Name=abc

- If you don't use the `this` keyword, compiler automatically adds this keyword while invoking the method.

```
class demo
{
    void method1()
    {
        System.out.println("I am method 1");
    }

    void method2()
    {
        System.out.println("I am method 2");
        this.method1();
    }
}
```

```
class sample
{
    public static void main(String args[])
    {
        demo d=new demo();
        d.method2();
    }
}
```

Output

I am method 2
I am method 1

Constructor

- It is used to reuse the constructor.

```
class demo
{
    demo()
    {
        System.out.println("I am demo constructor");
    }

    demo(int a)
    {
        this();
        System.out.println("A="+a);
    }
}
```

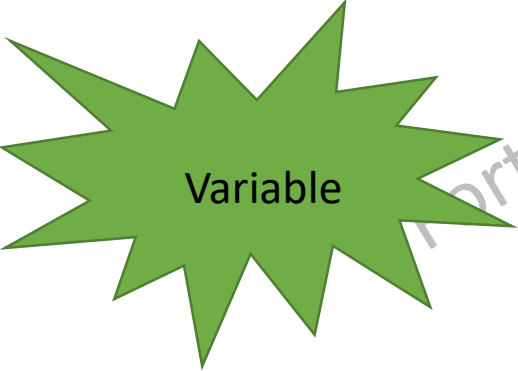
```
class sample
{
    public static void main(String args[])
    {
        demo d=new demo(10);
    }
}
```

Output

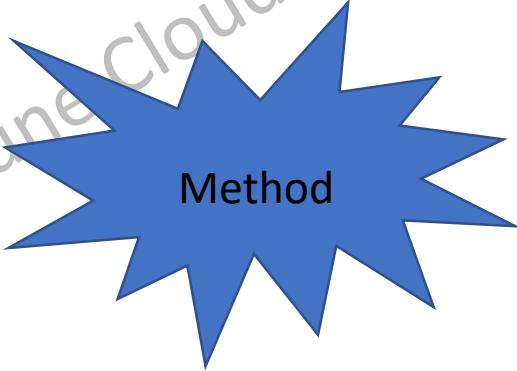
I am demo constructor
A=10

Static

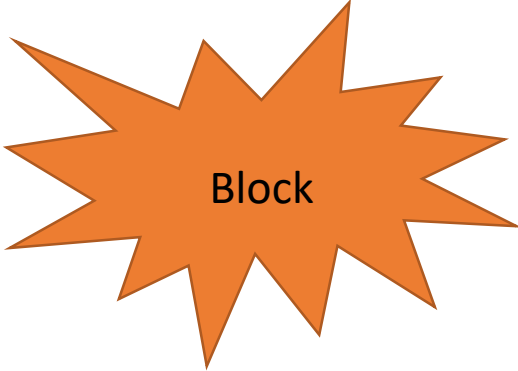
- **static keyword** is mainly used for memory management.
- static is used for a constant variable or a method that is same for every instance of a class.
- When a member of the class is declared as static, it can be accessed before the objects of its class are created, and without any object reference.



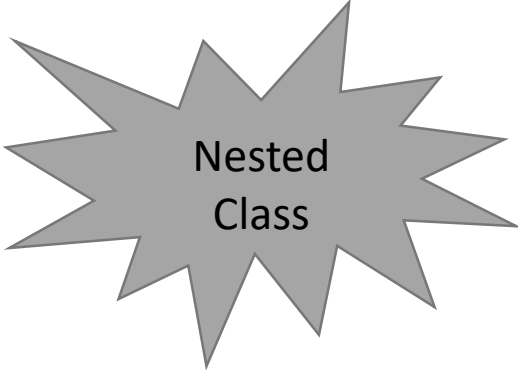
Variable



Method



Block



Nested
Class



Static Variable

- The static variable can be used to refer to the common property of all objects.
- The static variable gets memory only once in the class area at the time of class loading.

```
class Emp_stat
{
    int id;
    String name;
    static String company_name="Wipro";

    Emp_stat(int id,String name)
    {
        this.id=id;
        this.name=name;
    }

    public void emp_details()
    {
        System.out.println("Id="+id);
        System.out.println("Name="+name);
        System.out.println("Company name="+company_name);
    }
}
```

```
public class static_variable
{
    public static void main(String[] args)
    {
        Emp_stat e=new Emp_stat(1,"abc");
        e.emp_details();

        Emp_stat e1=new Emp_stat(2,"pqr");
        e1.emp_details();
    }
}
```

Static Method

- A static method can be invoked without the need for creating an instance of a class.
- A static method can access static data member and can change the value of it.

```
class Abcd
{
    int id;
    String name;
    static String company_name="Wipro";

    static void display()
    {
        System.out.println("I am static method");

        company_name="amdocs";
    }

    Abcd(int id,String name)
    {
        this.id=id;
        this.name=name;
    }
}
```

```
public void emp_details()
{
    System.out.println("Id="+id);
    System.out.println("Name="+name);
    System.out.println("Company
name="+company_name);
}

public class static_variable
{
    public static void main(String[] args)
    {
        Abcd.display();

        Abcd a=new Abcd(1,"abcd");
        a.emp_details();
    }
}
```



Static Block

- Is used to initialize the static data member.
- It is executed before the main method at the time of classloading.

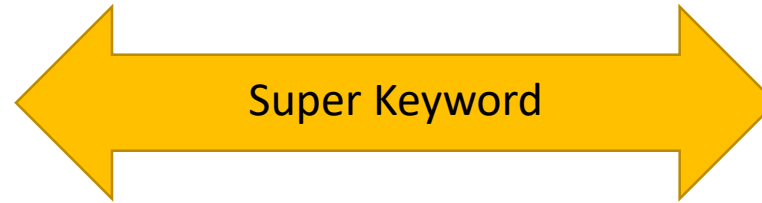
```
class A1
{
    static
    {
        System.out.println("I am static block");
    }

    public void display()
    {
        System.out.println("Display method");
    }
}

public class static_block {

    public static void main(String[] args) {

        A1 a=new A1();
        a.display();
    }
}
```



- The **super keyword** refers to superclass (parent) objects.
- It is used to call superclass methods, and to access the superclass constructor.
- The most common use of the **super keyword** is to eliminate the confusion between superclasses and subclasses that have methods with the same name.
- The **super** keyword in Java is a reference variable which is used to refer immediate parent class object.

Usage of Super Keyword

1

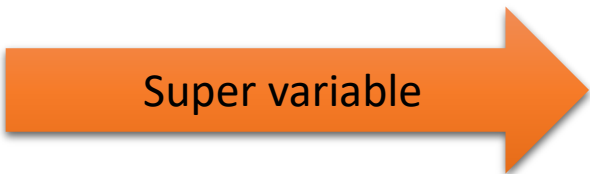
Super can be used to refer immediate parent class instance variable.

2

Super can be used to invoke immediate parent class method.

3

super() can be used to invoke immediate parent class constructor.



Super variable

- Access the data member or field of parent class
- It is used if parent class and child class have same fields.

Example

```
class demo
{
    String name="Abc";
}

class sample extends demo{

    String name="Xyz";

    void show()
    {
        System.out.println("Name="+super.name);
        System.out.println("Name="+name);
    }

    public static void main(String args[]) {
        sample s1=new sample();
        s1.show();
    }
}
```



Output



Name=Abc
Name=Xyz

Super Method

- Used to invoke parent class method
- It should be used if subclass contains the same method as parent class.
- It is used if method is overridden

Example



```
class demo
{
    void show()
    {
        System.out.println("Super class method");
    }
}

class sample extends demo{
    void show()
    {
        super.show();
        System.out.println("Sub class method");
    }
}

public static void main(String args[]) {
    sample s1=new sample();
    s1.show();
}
```

Output



Super class method
Sub class method

Super Constructor

- Used to invoke the parent class constructor.

Note: `super()` is added in each class constructor automatically by compiler if there is no `super()`

Example

```
class demo
{
    demo()
    {
        System.out.println("Super constructor");
    }
}

class sample extends demo{

    sample()
    {
        super();
        System.out.println("Sub class constructor");
    }

    public static void main(String args[]) {
        sample s1=new sample();
    }
}
```

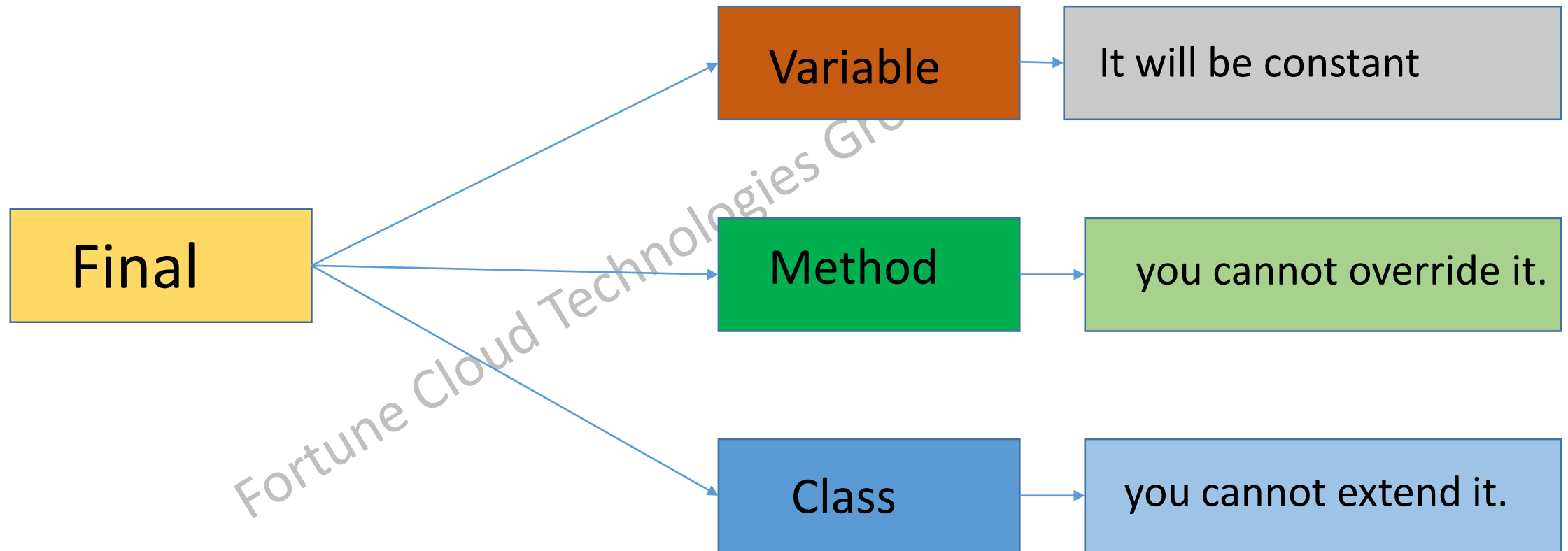


Output



Super constructor
Sub class constructor

Final Keyword



- If you make any variable as final, you cannot change the value of final variable(It will be constant)

```
public class sample
{
    final int a=10;//final variable
    void display()
    {
        a=20; //error
    }

    public static void main(String[] args)
    {
        sample s=new sample();
        s.display();
    }
}
```

Final Method

- If you make any method as final, you cannot override it.

```
class Parent
{
    final void display()
    {
        System.out.println("final method");
    }
}
```

```
public class sample extends Parent
{
    void display()
    {
        System.out.println("final method");
    }

    public static void main(String[] args)
    {
        sample s=new sample();
        s.display();
    }
}
```

Final Class

- If you make any class as final, you cannot extend it.

```
final class Parent
{ }

public class sample extends Parent
{
    public static void main(String[] args)
    {
        System.out.println("Hello world");
    }
}
```

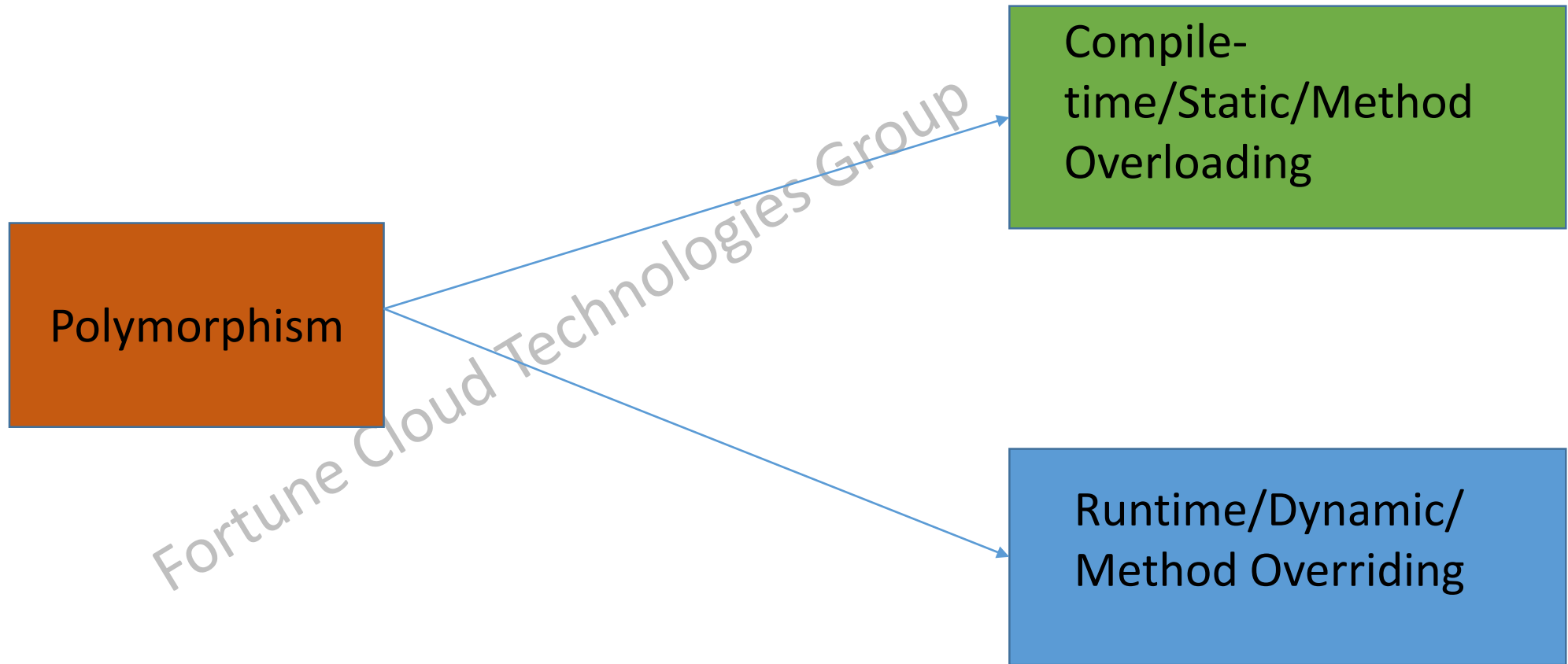
Polymorphism



- ❑ **Polymorphism in Java** is a concept by which we can perform a *single action in different ways*.
- ❑ Greek words: "poly" means many and "morphs" means forms.
- ❑ Perform a single action in different ways.
- ❑ One thing can behave or perform differently in different situation.

Example





Method Overloading

- If a class has multiple methods having same name but different in parameters, it is known as **Method Overloading**.

2 ways (same class)

```
graph TD; A[2 ways (same class)] --> B[By changing number of arguments]; A --> C[By changing the data type];
```

By changing number of arguments

By changing the data type

Method Overloading is not possible by changing the **return type** of the **method** only.

Example

```
Class Demo{
```

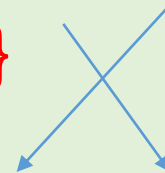
```
int sum(int a , int b){ return a;}
```

```
int sum(float c , int d){ return c;}
```

```
void sum(float a,float b){}
```

```
float sum(float a,float b){return a;}  
}
```

Change Data
type



Method Overriding

- If subclass (child class) has the same method as declared in the parent class, it is known as **method overriding in Java**.

Rules for Java Method Overriding

- The method must have the same name as in the parent class
- The method must have the same parameter as in the parent class.
- There must be an IS-A relationship (inheritance).

Example

```
class calculate{  
    //method overring
```

```
    float run(int a,int c)  
    {  
        return c;  
    }  
}
```

```
class addition extends  
calculate{
```

```
    @Override  
    float run(int b,int d)  
    {  
        return d;  
    }  
}
```

Abstraction

- ❑ **Abstraction** is a process of **hiding the implementation details** and **showing only functionality** to the user.
- ❑ Another way, it shows only essential things to the user and hides the internal details.
- ❑ We can achieve Abstraction by using **ABSTRACT CLASS** and **INTERFACE**.
- ❑ EX. Sms ,ATM, Cell phone.

Abstraction



Fig: Realtime Example of Abstraction in Java

Rules

- An abstract class must be declared with an abstract keyword.
- It can have abstract and non-abstract methods.
- It cannot be instantiated.
- It can have constructors and static methods also.
- It can have final methods

Example 1

```
abstract class demo
{
    abstract void show();
}

class sample extends demo{
    void show()
    {
        System.out.println("Abstract class method");
    }

    public static void main(String args[])
    {
        sample s1=new sample();
        s1.show();
    }
}
```

Output



Abstract class method

Example 2

abstract class demo

```
{  
    abstract void payment();  
}
```

class wallet extends demo{

```
    void payment()  
    {  
        System.out.println("Wallet payment success");  
    }  
}
```

class netbanking extends demo{

```
    void payment()  
    {  
        System.out.println("Netbanking payment success");  
    }  
}
```

class sample

```
{  
    public static void main(String args[])  
    {  
        wallet w=new wallet();  
        w.payment();  
  
        netbanking n=new netbanking();  
        n.payment();  
    }  
}
```

Output



Wallet payment success
Netbanking payment success

abstract class VS interface

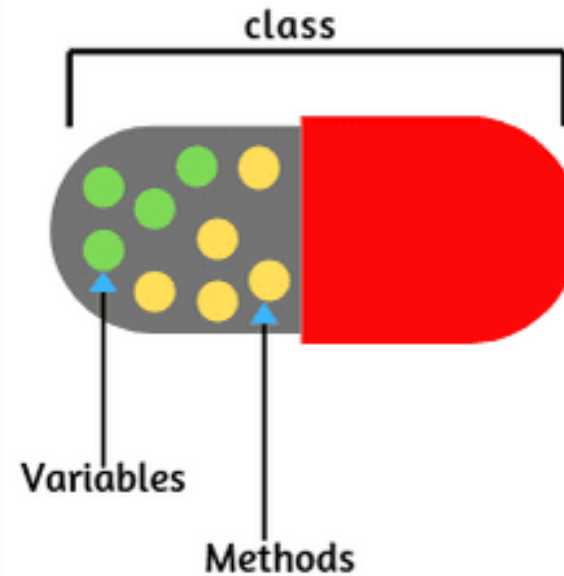
abstract	interface
abstract and non-abstract methods.	can have only abstract methods
can have final, non-final, static and non-static variables .	only static and final variables .
abstract class can be extended using keyword "extends".	interface can be implemented using keyword "implements".
A Java abstract class can have class members like private, protected, etc.	Members of a Java interface are public by default.
Example: <pre>public abstract class Shape{ public abstract void draw(); }</pre>	Example: <pre>public interface Shape{ void draw(); }</pre>

Encapsulation

- ❑ Encapsulation in Java is a ***process of wrapping code and data together into a single unit.***
- ❑ We can create a fully encapsulated class in Java by making **all the data members of the class private.**
- ❑ Now we can use setter and getter methods to set and get the data in it.

Example

```
class
{
    data members
    +
    methods (behavior)
}
```





- **Aggregation** is a term which is used to refer one way relationship between two objects
- In **Java**, **aggregation** represents HAS-A relationship, which means when a class contains reference of another class known to have **aggregation**.
- If a class have an entity reference, it is known as Aggregation.



When use Aggregation ?

- Code reuse is also best achieved by aggregation when there is no is-a relationship.

Example

```
public class Address
{
    String state;
    String city;

    public Address(String state , String city)
    {
        this.state = state;
        this.city = city;
    }
}
```

```
public class Employee
{
    String name;
    Address address;

    public Emp(String name, Address address) {
        this.name = name;
        this.address=address;
    }

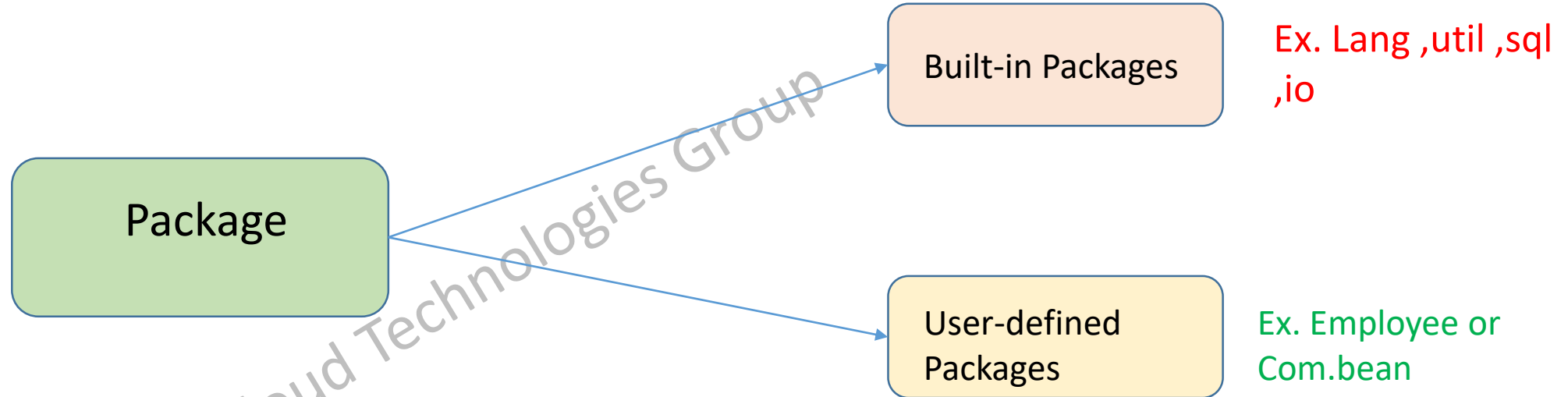
    void display()
    {
        System.out.println(name);
        System.out.println(address.state);
        System.out.println(address.city);
    }
}
```

```
public static void main(String[] args)
{
    Address a1=new Address("MH","Pune");

    Employee e1=new Employee("Abc",a1);

    e1.display();
}
}
```

Package



- A package in Java is used to group related classes
- **a folder in a file directory.**
- Java package is used to categorize the classes and interfaces so that they can be easily maintained.

How to access package from another package?

3 ways

`import package.*;`

`Import package.classname;`

`fully qualified name.`

Import package.*

```
package pack1;  
  
public class test1  
{  
    public void show()  
    {  
        System.out.println("pack1 – test1");  
    }  
}
```

```
package pack2;  
import pack1.*;  
  
class demo  
{  
    public static void main(String args[])  
    {  
        test1 t1= new test1();  
        t1.show();  
    }  
}
```


Import package.classname

```
package pack1;

public class test1
{
    public void show()
    {
        System.out.println("pack1 – test1");
    }
}

public class test2
{
    public void display()
    {
        System.out.println("pack1 – test2");
    }
}
```

```
package pack2;
import pack1.test1;

class demo
{
    public static void main(String args[])
    {
        test1 t1= new test1();
        t1.show();
    }
}
```

Fully qualified name

```
package pack1;

public class test1
{
    public void show()
    {
        System.out.println("pack1 – test1");
    }
}
```

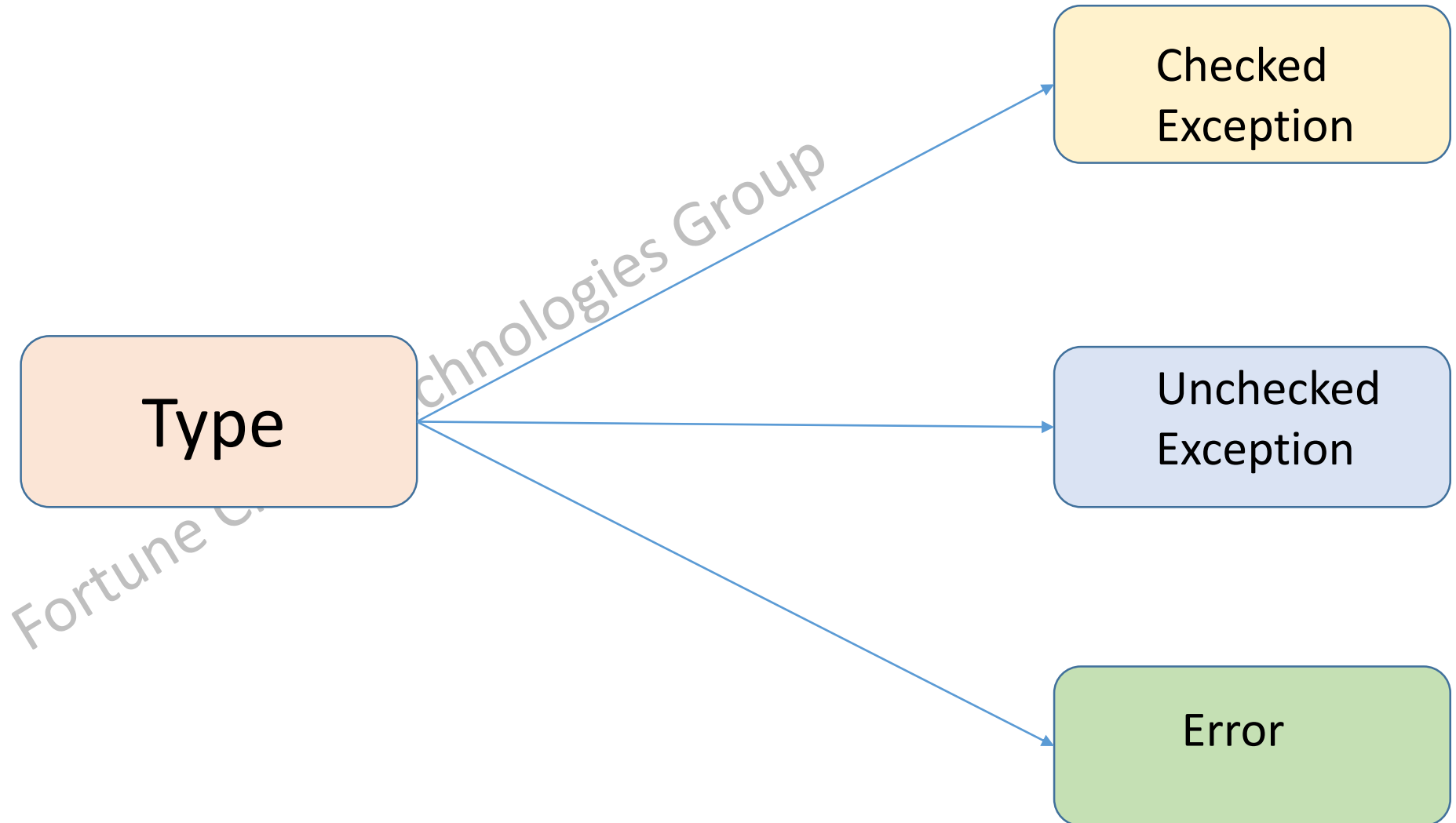
```
package pack2;

class demo
{
    public static void main(String args[])
    {
        pack1.test1 t1= new pack1.test1();
        t1.show();
    }
}
```

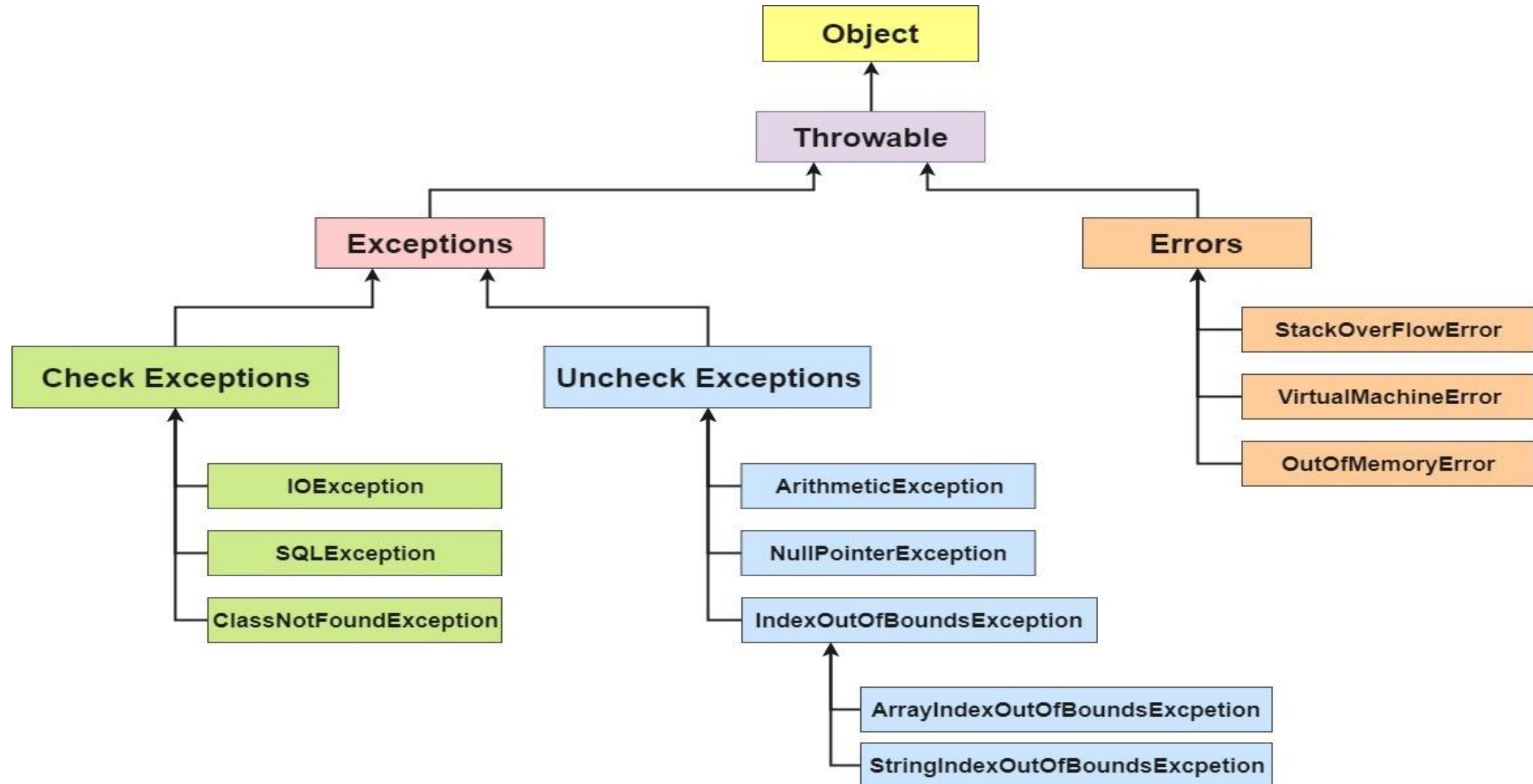
Exception Handling

- The **Exception Handling in Java** is one of the powerful *mechanism to handle the runtime errors* so that normal flow of the application can be maintained.
- In Java, an exception is an event that disrupts the normal flow of the program.
- It is an object which is thrown at runtime.
- Exception Handling is a mechanism to handle runtime errors such as `ClassNotFoundException`, `IOException`, `SQLException` etc.

Types of Java Exceptions

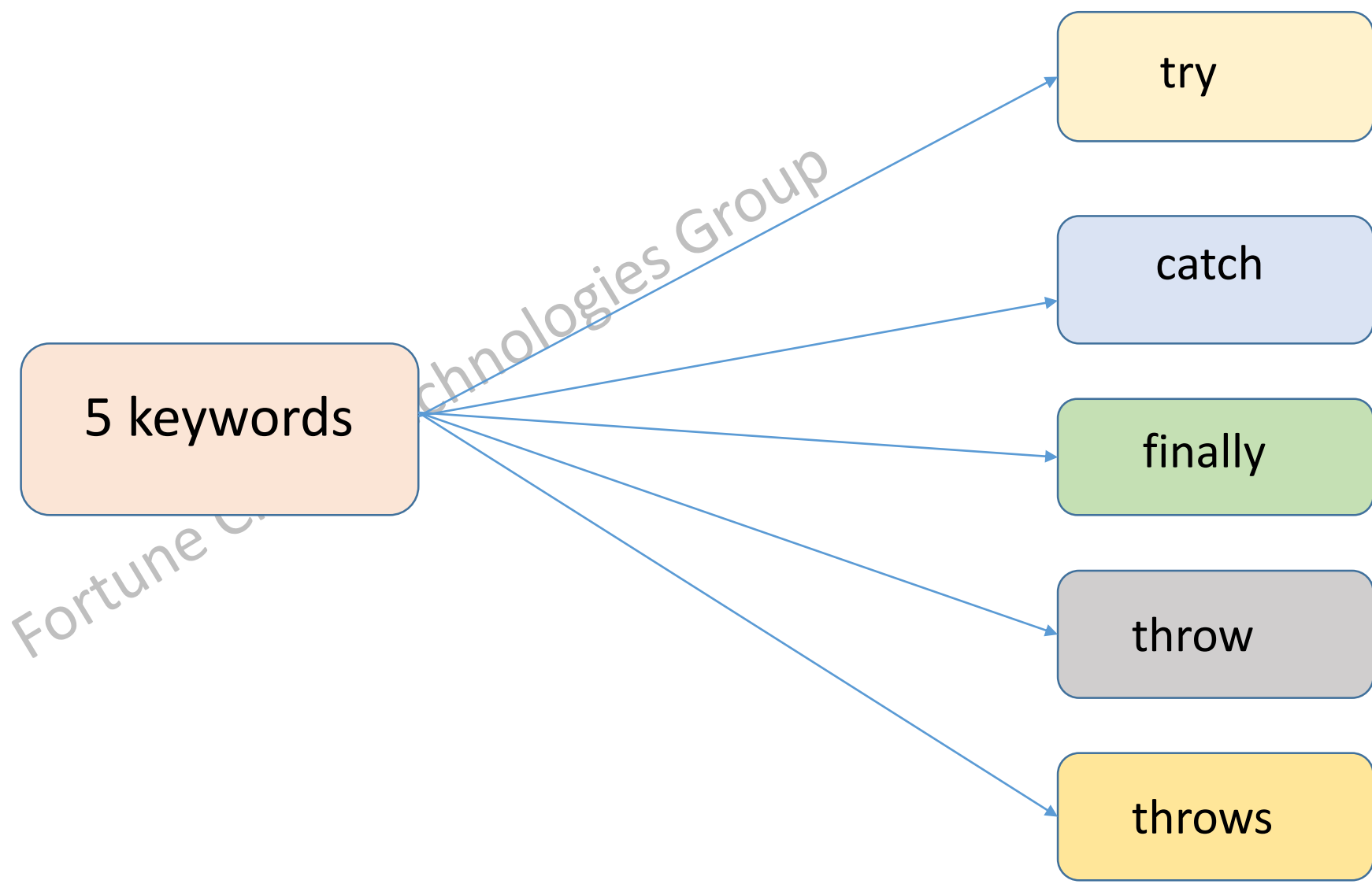


Hierarchy of Java Exception classes



Java Exception Keywords

5 keywords



```
graph LR; A[5 keywords] --> B[try]; A --> C[catch]; A --> D[finally]; A --> E[throw]; A --> F[throws];
```

try

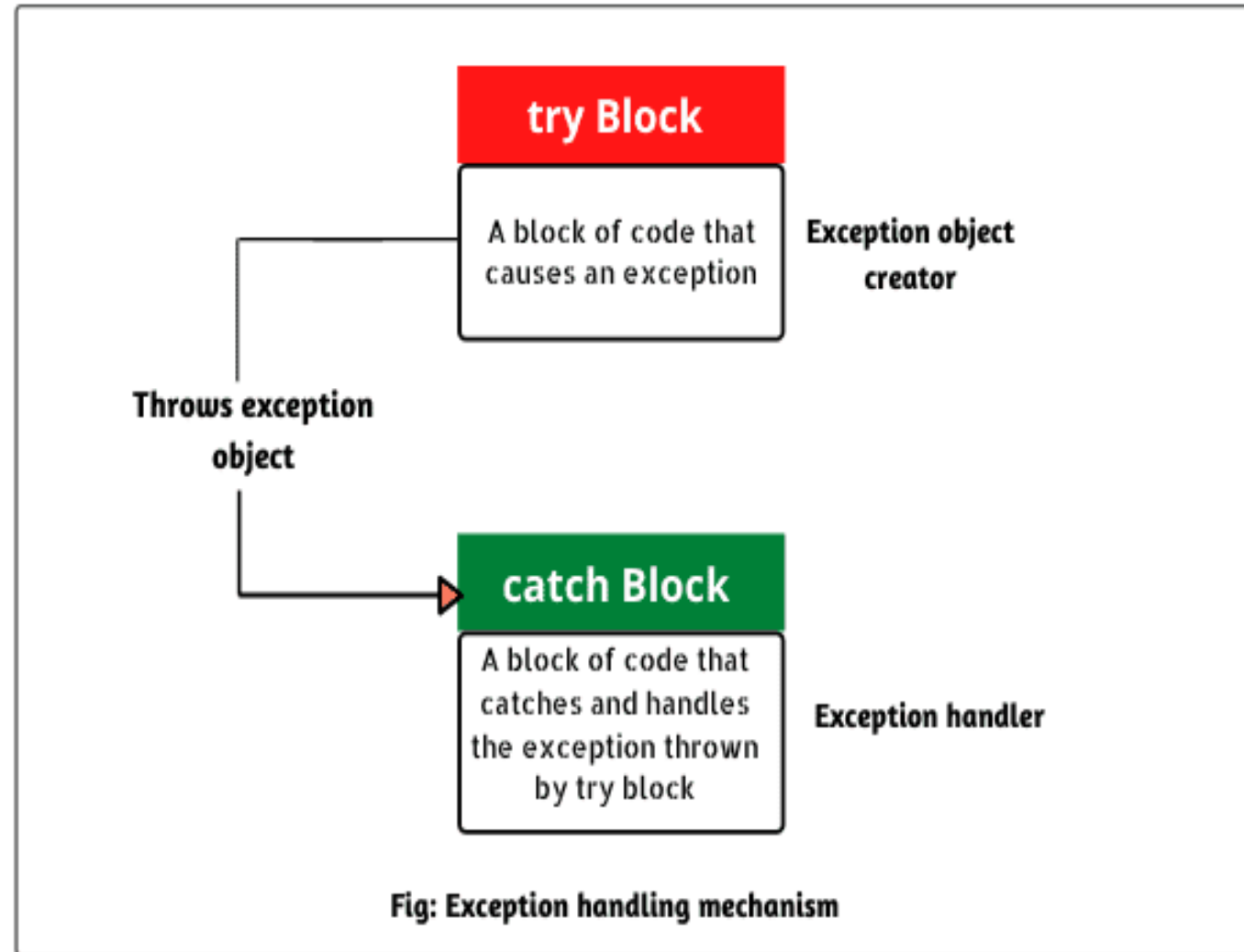
catch

finally

throw

throws

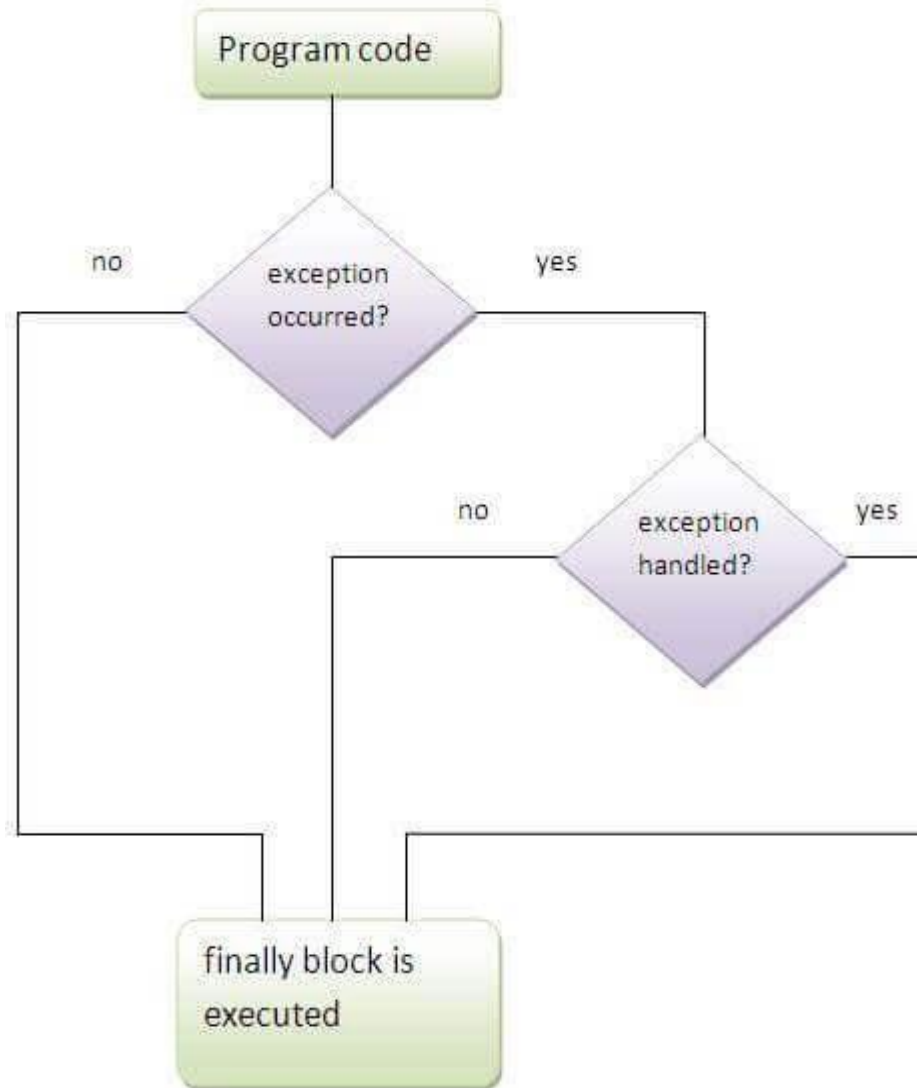
try & catch blocks



Example

```
public class Demo{  
    public static void main(String args[]){  
        try{  
  
            int data=5/0;  
        }  
  
        catch(ArithmeticException e){  
  
            System.out.println(e);  
        }  
        System.out.println("Last line");  
    }  
}
```


finally block



- Java finally block is always executed whether exception is handled or not.
- Java finally block follows try or catch block.

Example

```
class Demo{  
    public static void main(String args[]){  
        try  
        { }  
        catch()  
        { }  
        finally  
        {  
            System.out.println("finally block is always executed");  
        }  
    }  
}
```

throw keyword

- The **throw** keyword in **Java** is used to explicitly **throw** an exception from a method or any block of code.
- The **throw** keyword is mainly used to **throw** custom exceptions.

```
public class throw_keyword {  
    void check(int num)  
    {  
        if(num>=0)  
        {  
            throw new ArithmeticException("Number is positive");  
        }  
        else  
        {  
            System.out.println("Number is negative");  
        }  
    }  
  
    public static void main(String[] args)  
    {  
        throw_throws t1=new throw_throws();  
        t1.check(-10);  
    }  
}
```

Example 2

```
public class throw_keyword {  
    static void check(int num)  
    {  
        if(num>=0)  
        {  
            throw new ArithmeticException("Number is positive");  
        }  
        else  
        {  
            System.out.println("Number is negative");  
        }  
    }  
  
    public static void main(String[] args)  
    {  
        check(-10);  
    }  
}
```



throws keyword

- The **throws keyword** is used to declare which exceptions can be **thrown** from a method
- while the **throw keyword** is used to explicitly **throw** an exception within a method or block of code.
- The **throws keyword** is used in a method signature and declares which exceptions can be **thrown** from a method.
- It gives an information to the programmer that there may occur an exception so it is better for the programmer to provide the exception handling code so that normal flow can be maintained.

Example

Class demo

```
{  
    void show() throws IOException  
    {  
        throw new IOException("test");  
    }  
}
```

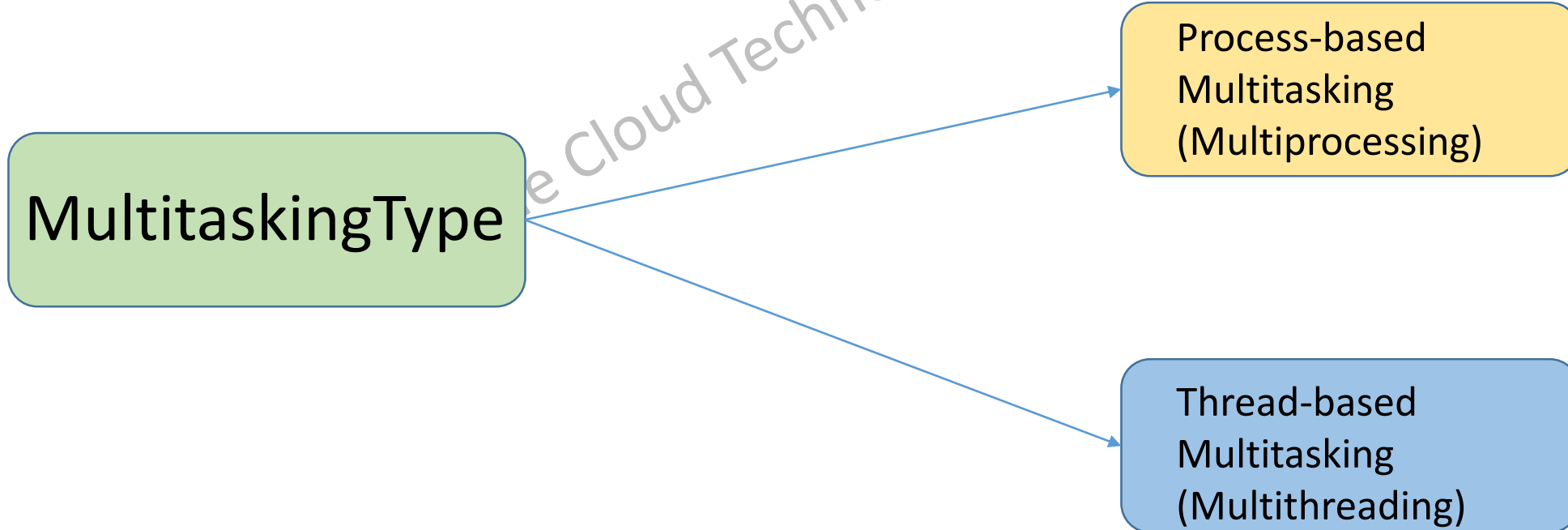
```
public class sample  
{  
    public static void main(String args[]){  
        try  
        {  
            demo d=new demo();  
            d.show();  
        }  
        catch(Exception e)  
        {  
            System.out.println("exception handled");  
        }  
    }  
}
```

throw vs throws

No.	throw	throws
1)	Java throw keyword is used to explicitly throw an exception.	Java throws keyword is used to declare an exception.
2)	Throw is followed by an new instance.	Throws is followed by class.
3)	Throw is used within the method.	Throws is used with the method signature.
4)	You cannot throw multiple exceptions.	You can declare multiple exceptions e.g. public void method()throws IOException,SQLException

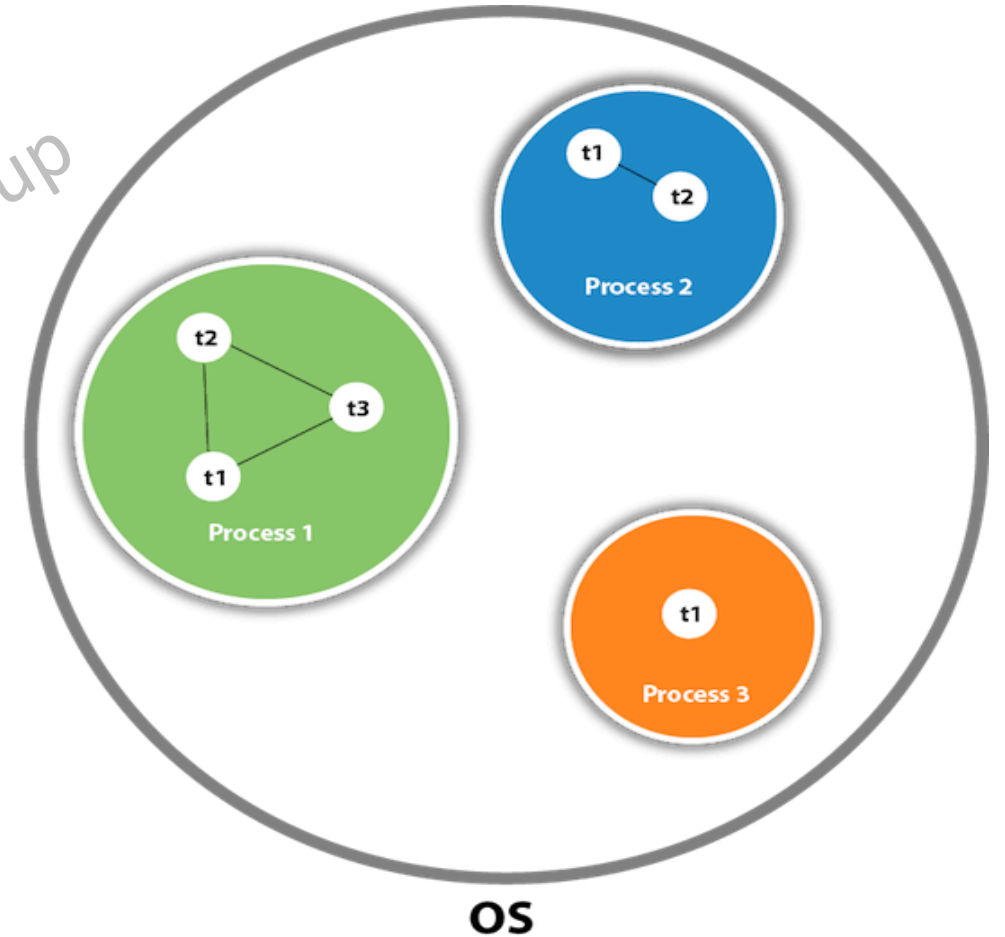
Multithreading

- Multithreading in Java is a process of executing multiple threads simultaneously.
- A thread is a lightweight sub-process, the smallest unit of processing.
- Java Multithreading is mostly used in games, animation, etc.



What is Thread in java

- A thread is a **lightweight subprocess**, the **smallest unit of processing**. It is a separate path of execution.
- Threads are **independent**. If there occurs exception in one thread, it doesn't affect other threads.
- It uses a **shared memory area**.



Life Cycle of a Thread

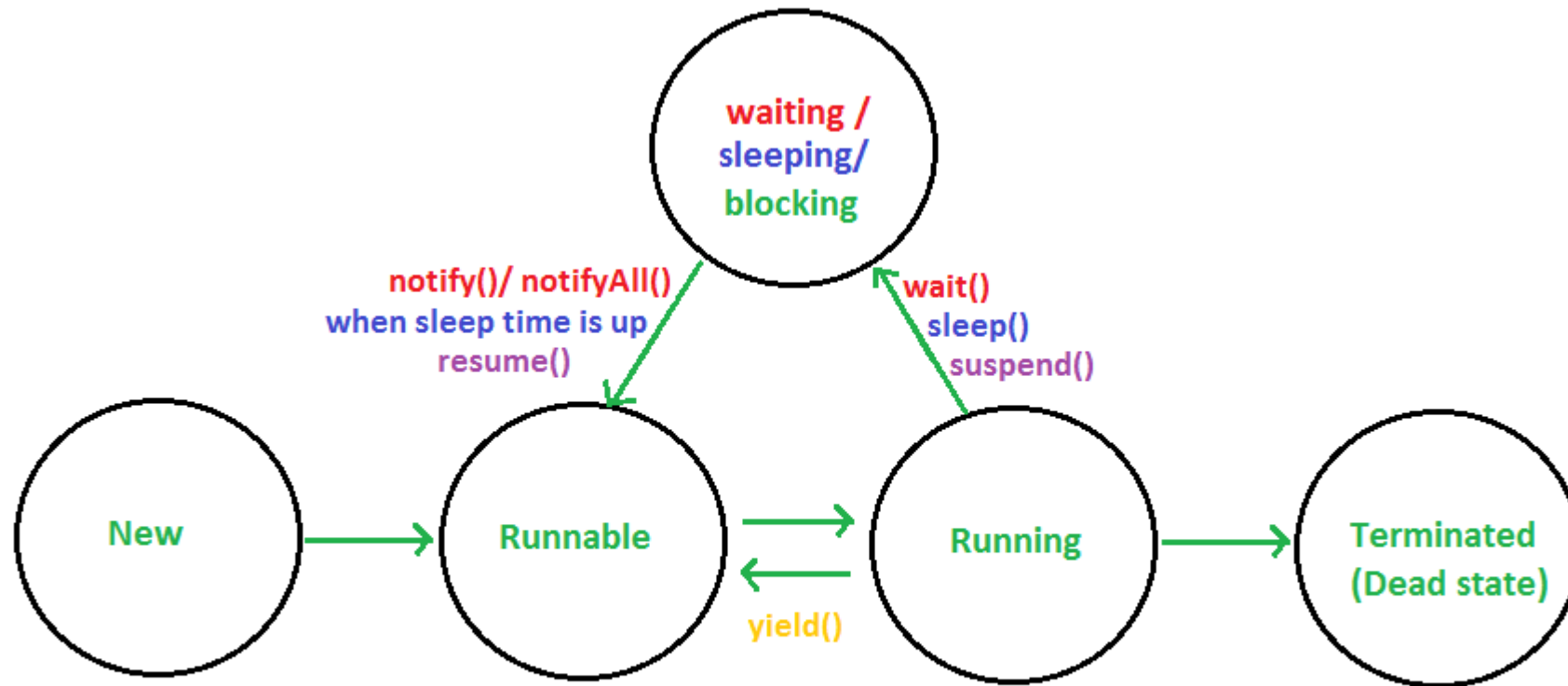
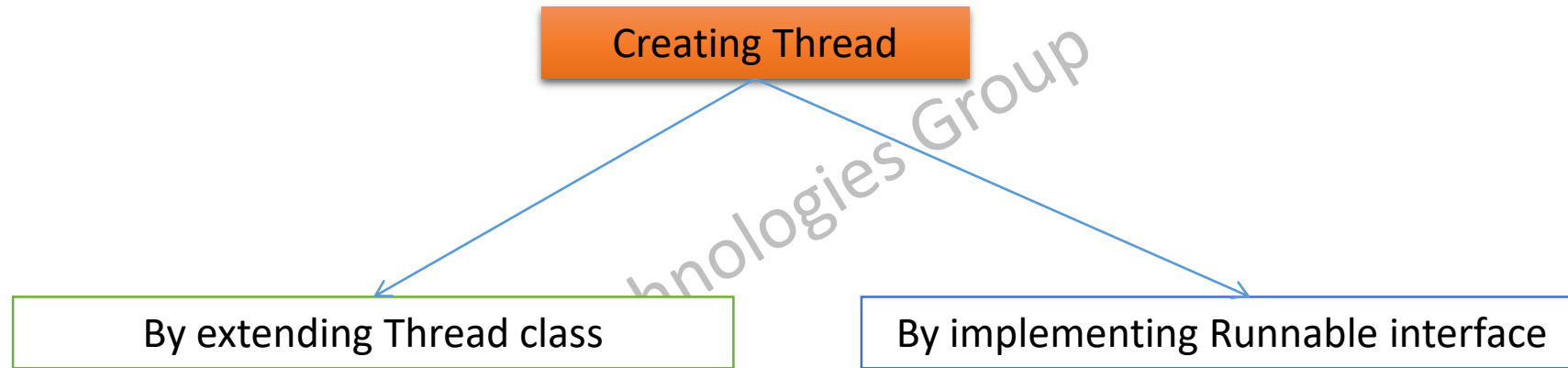


Fig. THREAD STATES



By extending Thread class

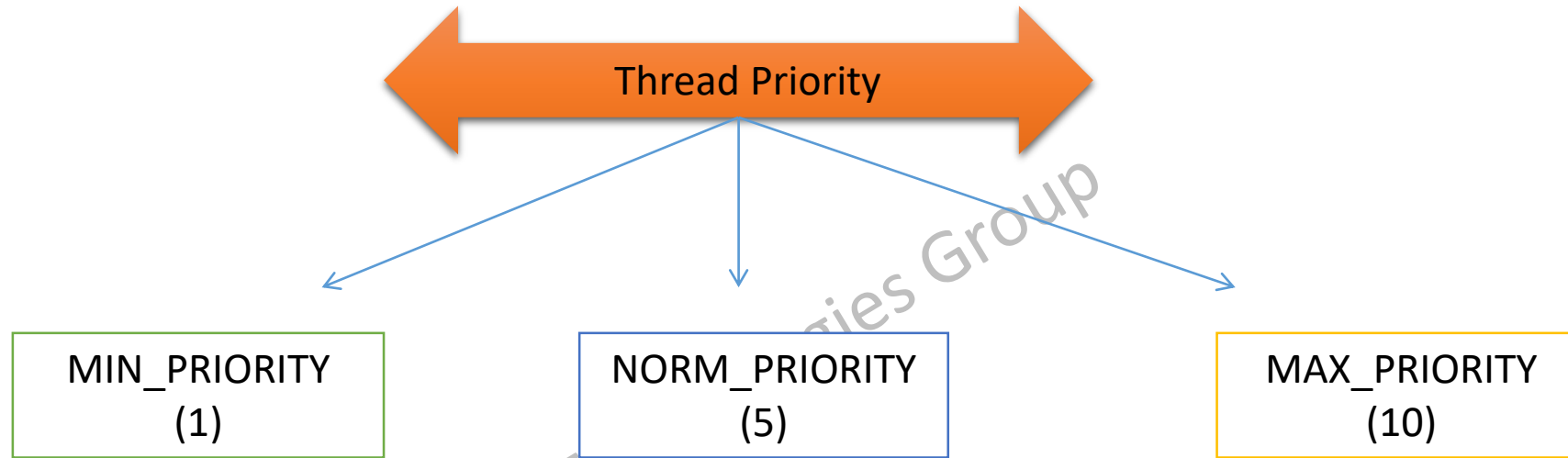
```
public class sample extends Thread
{
    public void run()
    {
        System.out.println("Thread is running");
    }

    public static void main(String[] args)
    {
        sample s=new sample();
        s.start();
    }
}
```

By implementing Runnable interface

```
public class sample implements Runnable
{
    public void run()
    {
        System.out.println("Thread is running");
    }

    public static void main(String[] args)
    {
        sample s=new sample();
        Thread t1 =new Thread(s);
        t1.start();
    }
}
```



```
class sample extends Thread{
    public void run()
    {
        System.out.println("Thread name:"+Thread.currentThread().getName());
        System.out.println("Thread priority:"+Thread.currentThread().getPriority());
    }

    public static void main(String args[]) {

        sample m1=new sample();
        sample m2=new sample();

        m1.setPriority(Thread.MIN_PRIORITY);
        m2.setPriority(Thread.MAX_PRIORITY);

        m1.start();
        m2.start();
    }
}
```

- The **sleep() method** is a **static method** of Thread class and it makes the thread **sleep**/stop working for a specific amount of time.
- The **sleep() method** throws an InterruptedException if a thread is interrupted by other threads, that means Thread.
- The sleep() method of Thread class is used to sleep a thread for the specified amount of time.

Example

```
public class thread_sleep extends Thread
{
    public void run()
    {
        for(int i=1;i<=10;i++)
        {
            try
            {
                Thread.sleep(1000);
            }
            catch(Exception e)
            {
                System.out.println(e);
            }

            System.out.println(i);
        }
    }
}
```

```
public static void main(String[] args)
{
    thread_sleep t1=new thread_sleep();
    thread_sleep t2=new thread_sleep();

    t1.start();
    t2.start();
}
}
```

- The Thread class provides methods to change and get the name of a thread.

```
public class thread_rename extends Thread
{
    public void run()
    {
        System.out.println("Run method");
    }

    public static void main(String[] args)
    {
        thread_rename t1=new thread_rename();

        t1.start();
        t1.setName("Thread1");

        System.out.println("Thread name="+t1.getName());
    }
}
```

- A **ThreadGroup** represents a set of threads.
- Thread groups provide a mechanism for collecting multiple threads into a single object and manipulating those threads all at once, rather than individually.
- For example, you can start or suspend all the threads within a group with a single method call.
- Java thread group is implemented by **java.lang.ThreadGroup** class.

Example

```
public class thread_group implements Runnable
{
    public void run()
    {
        System.out.println("Run method");
    }
}
```

```
public static void main(String[] args)
{
    thread_group tg=new thread_group();
    ThreadGroup group=new ThreadGroup("All threads");

    Thread t1=new Thread(group,tg,"thread1");
    t1.start();
    Thread t2=new Thread(group,tg,"thread2");
    t2.start();
    Thread t3=new Thread(group,tg,"thread3");
    t3.start();

    System.out.println("Group name="+group.getName());
    group.list();
}
}
```

- The **yield()** method of thread class causes the currently executing thread object to temporarily pause and allow other threads to execute.
- This method does not return any value.
- A **yield() method** is a static **method** of Thread class and it can stop the currently executing thread and will give a chance to other waiting threads of the same priority.
- If in case there are no waiting threads or if all the waiting threads have low priority then the same thread will continue its execution.
- The advantage of **yield()** method is to get a chance to execute other waiting threads so if our current thread takes more time to execute and allocate processor to other threads.

Example

```
public class sample extends Thread
{
    public void run()
    {
        for(int i=0; i<5 ; i++)
        {
            System.out.println(Thread.currentThread().getName() + " in control");
        }
    }
}
```

```
public static void main(String[]args)
{
    sample t1 = new sample();
    sample t2 = new sample();

    t1.start();
    t2.start();
    for (int i=0; i<5; i++)
    {
        t1.yield();
        System.out.println("yeild"+Thread.currentThread().getName() + " in control");
    }
}
```

Difference between **sleep()** & **yield()** method

- **sleep() method** takes duration as argument and ensures that the current thread suspends its execution for the specified time.
 - **yield() method** is merely a hint to the scheduler that the current thread is willing to yield its current use of a processor. Scheduler can even ignore this hint to yield.
-
- When **sleep() method** is called current thread will definitely go to sleep for the specified time.
 - In case of **yield() method** first of all it is just a hint which can be ignored, even if the current thread yields the CPU it can start running again if there is no other thread with the same or more thread priority.
-
- **sleep() method** throws InterruptedException if sleeping thread is interrupted so call to sleep() method should either be enclosed in try-catch block or it should be declared using throws clause.
 - **yield() method** doesn't throw InterruptedException.

- A **shutdown hook** is simply an initialized but unstarted thread.
- When the virtual machine begins its **shutdown** sequence it will start all registered **shutdown hooks** in some unspecified order and let them run concurrently
- We **can use** `java.lang.Runtime.addShutdownHook(Thread t)` **method** to **add a shutdown hook** in the JVM.
- Eg. user presses ctrl+c on the command prompt, `System.exit(int)` method is invoked, user logoff

Example

```
public class shutdown_hook
{
    static class Message extends Thread
    {
        public void run()
        {
            System.out.println("Bye.");
        }
    }
}
```

```
public static void main(String[] args) {
    try
    {
        Runtime.getRuntime().addShutdownHook(new Message());

        System.out.println("Program is starting...");

        System.out.println("Waiting for 3 seconds...");
        Thread.sleep(3000);

        System.out.println("Program is closing...");
    }
    catch (Exception e)
    {
        e.printStackTrace();
    }
}
```



- Java Synchronization is better option where we want to allow only one thread to access the shared resource.

For example, if multiple threads try to write within a same file then they may corrupt the data because one of the threads can override data or while one thread is opening the same file at the same time another thread might be closing the same file.

- Synchronization is built around an internal entity known as the lock or monitor. Every object has an lock associated with it.

Example

```
class demo
{
    synchronized void show(int n)
    {
        for(int i=1;i<=5;i++)
        {
            System.out.println(n*i);
            try
            {
                Thread.sleep(400);
            }
            catch(Exception e)
            {
                System.out.println(e);
            }
        }
    }
}
```

```
class MyThread1 extends Thread
{
    demo d;

    MyThread1(demo d)
    {
        this.d=d;
    }

    public void run()
    {
        d.show(5);
    }
}
```

```
class MyThread2 extends Thread
{
    demo d;

    MyThread2(demo d)
    {
        this.d=d;
    }

    public void run()
    {
        d.show(100);
    }
}
```

```
public class sample
{
    public static void main(String args[])
    {
        demo d1 = new demo();
        MyThread1 t1=new MyThread1(d1);
        MyThread2 t2=new MyThread2(d1);

        t1.start();
        t2.start();
    }
}
```

- A nested **class** that doesn't have any name is known as an **anonymous class**.

```
class demo
{
    synchronized void show(int n)
    {
        for(int i=1;i<=5;i++)
        {
            System.out.println(n*i);
            try
            {
                Thread.sleep(400);
            }
            catch(Exception e)
            {
                System.out.println(e);
            }
        }
    }
}
```

```
public class sample
{
    public static void main(String args[])
    {
        demo d1 = new demo();

        Thread t1=new Thread()
        {
            public void run()
            {
                d1.show(5);
            }
        };
    }
};
```

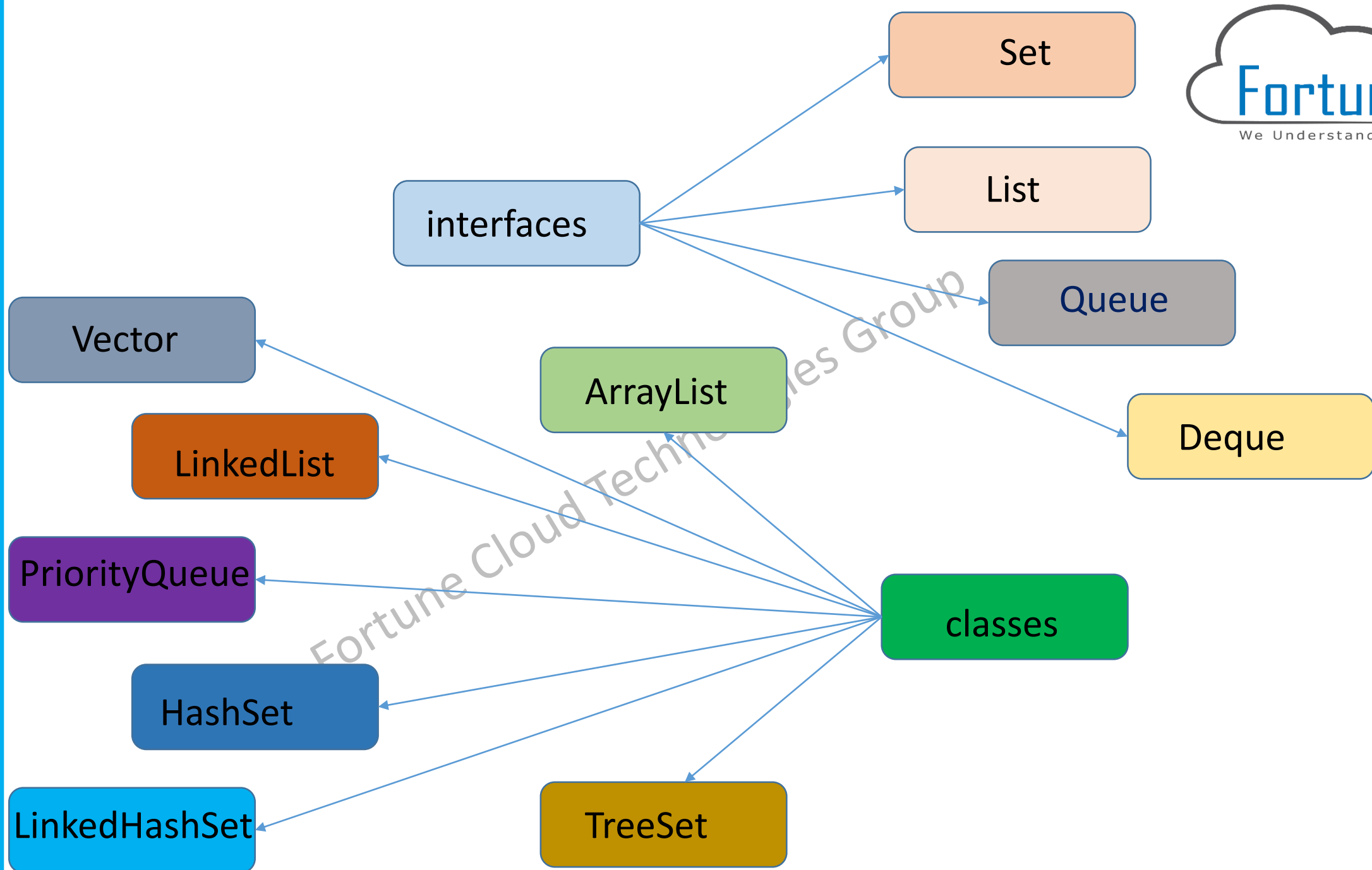
```
Thread t2=new Thread()
{
    public void run()
    {
        d1.show(100);
    }
};

t1.start();
t2.start();
}
```

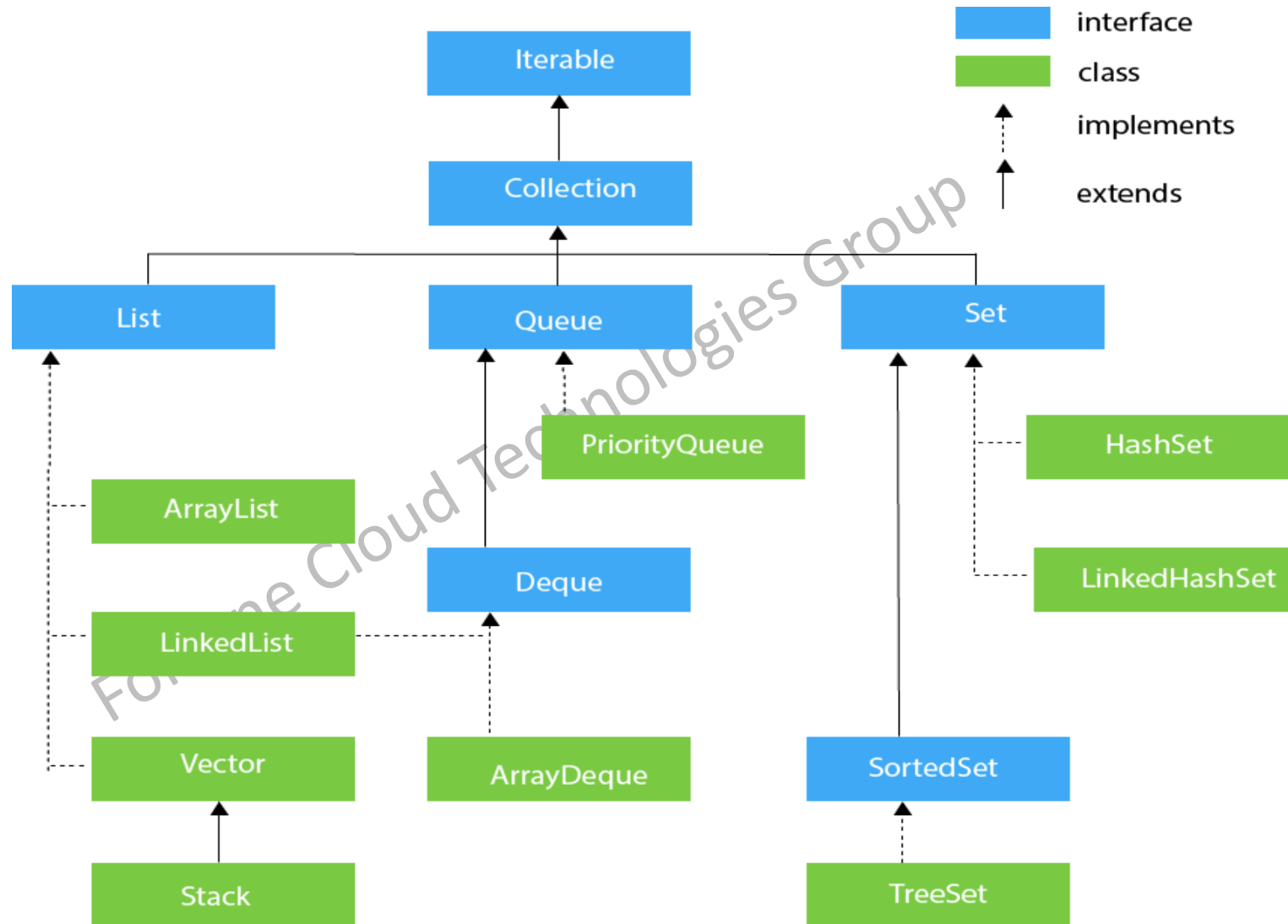

Collections



- **Collection in Java** is a framework that provides an architecture to **store and manipulate the group of objects.**
- Java Collection means **a single unit of objects.**
- perform on a data such as **searching, sorting, insertion, and deletion.**
- The **java.util** package contains all the **classes and interfaces** for the Collection framework.



Hierarchy of Collection Framework



- Array lists are created with an initial size. When this size is exceeded, the collection is automatically enlarged.
- **ArrayList** class uses a dynamic array for storing the elements. It is like an array, but there is no size limit.



- ArrayList class can contain duplicate elements.
- ArrayList class maintains insertion order.
- ArrayList class is non synchronized.
- ArrayList allows random access because array works at the index basis.
- manipulation is little bit slower than the LinkedList because a lot of shifting needs to occur if any element is removed from the array list.

Example

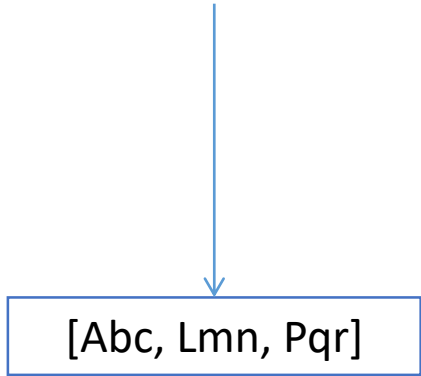
```
import java.util.*;

public class sample
{
    public static void main(String args[])
    {
        ArrayList<String> list=new ArrayList<String>();

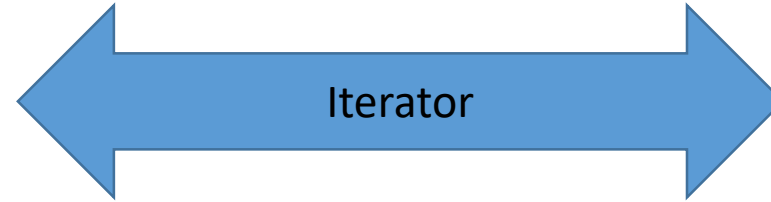
        list.add("Abc");
        list.add("Lmn");
        list.add("Pqr");

        System.out.println(list);
    }
}
```

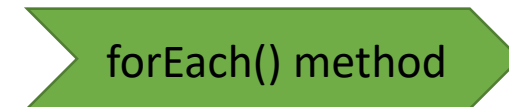
Output



[Abc, Lmn, Pqr]



- An **Iterator** is an object that can be used to loop through collections, like ArrayList and HashSet.
- It is called an "**iterator**" because "**iterating**" is the technical term for looping.
- To use an **Iterator**, you must import it from the **java**.



LinkedList

- **LinkedList should** be **used** where modifications to a collection are frequent like addition/deletion operations.
- **LinkedList** is much faster as compare to ArrayList in such cases.



- LinkedList class can contain duplicate elements.
- LinkedList class maintains insertion order.
- Manipulation is fast because no shifting needs to occur.

Example

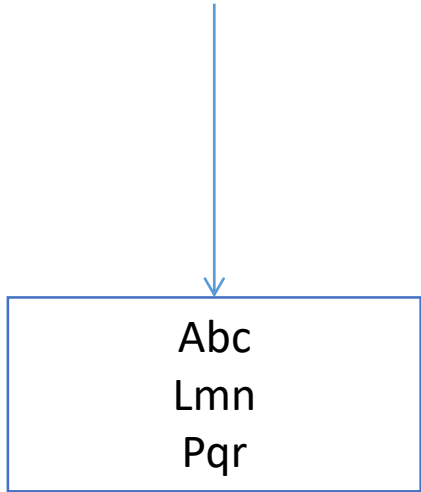
```
import java.util.*;

public class sample
{
    public static void main(String args[])
    {
        LinkedList<String> list=new LinkedList<String>();

        list.add("Abc");
        list.add("Lmn");
        list.add("Pqr");

        Iterator<String> itr=list.iterator();
        while(itr.hasNext())
        {
            System.out.println(itr.next());
        }
    }
}
```

Output



Abc
Lmn
Pqr

- In **Java**, a **list** interface is an ordered collection of objects in which duplicate values can be stored.
- **List** in Java provides the facility to maintain the *ordered collection*.
- We can also store the null elements in the list.
- The List interface is found in the java.util package and inherits the Collection interface.

Example

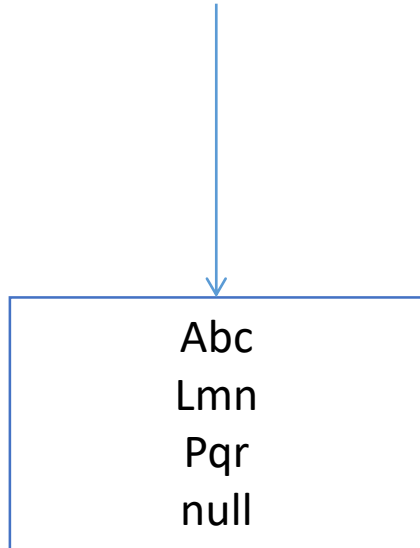
```
import java.util.*;

public class sample
{
    public static void main(String args[])
    {
        List<String> list=new ArrayList<String>();

        list.add("Abc");
        list.add("Lmn");
        list.add("Pqr");
        list.add(null);

        for(String str:list)
        {
            System.out.println(str);
        }
    }
}
```

Output



Abc
Lmn
Pqr
null

Convert Array to List

```
import java.util.*;

public class sample
{
    public static void main(String args[])
    {
        String a[]={"Abc","Lmn","Pqr","Xyz"};

        System.out.println("Array="+Arrays.toString(a));

        List<String> list=new ArrayList<String>();

        for(String str:a)
        {
            list.add(str);
        }

        System.out.println("List="+list);
    }
}
```

Output

Array=[Abc, Lmn, Pqr, Xyz]
List=[Abc, Lmn, Pqr, Xyz]

Convert List to Array

```
import java.util.*;

public class sample
{
    public static void main(String args[])
    {
        List<String> list=new ArrayList<String>();
        list.add("abc");
        list.add("lmn");
        list.add("pqr");

        String[] arr= list.toArray(new String[list.size()]);

        System.out.println("List="+list);
        System.out.println("Array="+Arrays.toString(arr));
    }
}
```

Output

List=[abc, lmn, pqr]

Array=[abc, lmn, pqr]

➤ **Get() and Set() method**

```
list.set(1,"qwerty");
```

```
list.get(1)
```

➤ **Collections.sort() method**

```
List<String> list=new ArrayList<String>();  
list.add("lmn");  
list.add("pqr");  
list.add("abc");  
  
Collections.sort(list);  
  
for(String str:list)  
{  
    System.out.println(str);  
}
```

Output

abc
lmn
pqr

- **Java HashSet** class is used to create a collection that uses a hash table for storage.
- The important points about **Java HashSet** class are: **HashSet** stores the elements by using a mechanism called hashing.
- **HashSet** contains unique elements only.
- In hashing, the informational content of a key is **used** to determine a unique value, called its hash code.



- HashSet stores the elements by using a mechanism called **hashing**.
- HashSet contains unique elements only.
- HashSet allows null value.
- HashSet doesn't maintain the insertion order.
- HashSet is the best approach for search operations.

Example

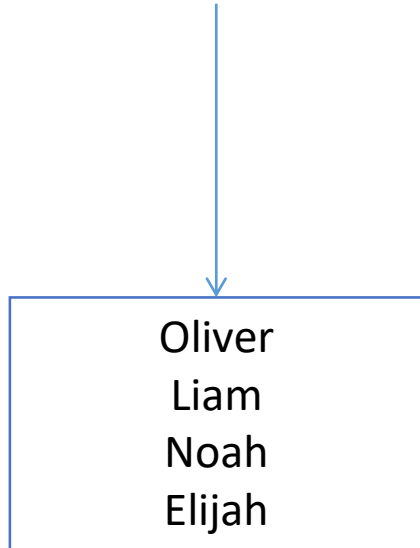
```
import java.util.*;

public class sample
{
    public static void main(String args[])
    {
        HashSet<String> s=new HashSet();

        s.add("Liam");
        s.add("Noah");
        s.add("Oliver");
        s.add("Elijah");

        Iterator<String> i=s.iterator();
        while(i.hasNext())
        {
            System.out.println(i.next());
        }
    }
}
```

Output



Oliver
Liam
Noah
Elijah

Notice, the elements iterate in an unordered collection.

- The **LinkedHashSet** class of the **Java collections framework** provides functionalities of both the hashtable and the linked list data **structure**.
- It implements the Set interface.
- Elements of **LinkedHashSet** are stored in hash tables similar to **HashSet**.



- LinkedHashSet class contains unique elements.
- LinkedHashSet class provides all optional set operation and permits null elements.
- LinkedHashSet class maintains insertion order.

Example

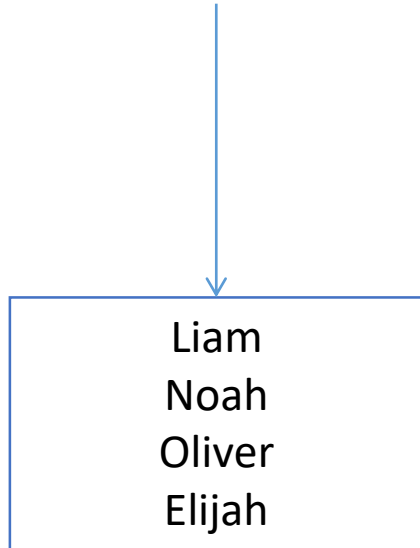
```
import java.util.*;

public class sample
{
    public static void main(String args[])
    {
        LinkedList<String> set=new LinkedList<>();

        set.add("Liam");
        set.add("Noah");
        set.add("Oliver");
        set.add("Elijah");

        Iterator<String> i=set.iterator();
        while(i.hasNext())
        {
            System.out.println(i.next());
        }
    }
}
```

Output



Liam
Noah
Oliver
Elijah

- **TreeSet** is one of the most important implementations of the SortedSet interface in **Java** that uses a Tree for storage.
- The ordering of the elements is maintained by a set using their natural ordering.
- The objects of the TreeSet class are stored in ascending order.



Important Points

- TreeSet class contains unique elements only like HashSet.
- TreeSet class access and retrieval times are quite fast.
- TreeSet class doesn't allow null element.
- TreeSet class maintains ascending order.

Example

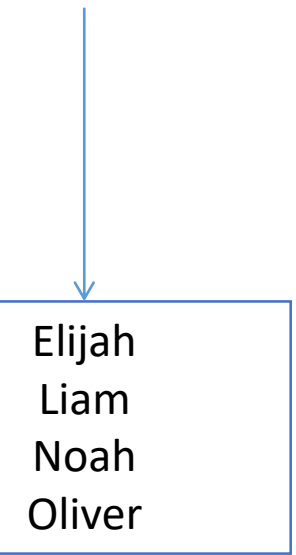
```
import java.util.*;

public class sample
{
    public static void main(String args[])
    {
        TreeSet<String> al=new TreeSet<String>();
        al.add("Liam");
        al.add("Noah");
        al.add("Oliver");
        al.add("Elijah");

        Iterator<String> itr=al.iterator();

        while(itr.hasNext())
        {
            System.out.println(itr.next());
        }
    }
}
```

Output



Elijah
Liam
Noah
Oliver

Map Interface

- A **map** contains values on the basis of key, i.e. key and value pair.
- Each key and value pair is known as an entry.
- A **Map** contains unique keys.
- A **Map** is useful if you have to search, update or delete elements on the basis of a key.



- A Map doesn't allow duplicate keys, but you can have duplicate values.
- HashMap and LinkedHashMap allow null keys and values, but TreeMap doesn't allow any null key or value.

Example

```
import java.util.*;

public class sample
{
    public static void main(String args[])
    {
        Map<Integer,String> map=new HashMap<Integer,String>();

        map.put(101,"Abc");
        map.put(102,"Lmn");
        map.put(103,"Pqr");

        for(Map.Entry m:map.entrySet())
        {
            System.out.println(m.getKey()+" "+m.getValue());
        }
    }
}
```

Output



```
101 Abc
102 Lmn
103 Pqr
```

HashMap

- It allows us to store the null elements as well, but there should be only one null key.
- **HashMap** class implements the Map interface which allows us to store key and value pair



- HashMap contains values based on the key.
- HashMap contains only unique keys.
- HashMap may have one null key and multiple null values.
- HashMap maintains no order.

Example

```
import java.util.*;

public class sample
{
    public static void main(String args[])
    {
        HashMap<Integer,String> map=new HashMap<Integer,String>();
        map.put(1,"Abc");
        map.put(2,"qwerty");
        map.put(3,"xyz");
        map.put(4,"lmn");

        for(Map.Entry m : map.entrySet())
        {
            System.out.println(m.getKey()+" "+m.getValue());
        }
    }
}
```

Output



```
1 Abc
2 qwerty
3 xyz
4 lmn
```

- The **Hashtable** class implements a **hash table**, which maps keys to values.
- Any non-null object can be **used** as a key or as a value.
- It is similar to HashMap, but is synchronized.



- A Hashtable is an array of a list. Each list is known as a bucket.
- The position of the bucket is identified by calling the hashCode() method.
- Hashtable class contains unique elements.
- Hashtable class doesn't allow null key or value.

Example

```
import java.util.*;

public class sample
{
    public static void main(String args[])
    {
        Hashtable<Integer,String> ht=new Hashtable<Integer,String>();

        ht.put(10,"Abc");
        ht.put(20,"Lmn");
        ht.put(30,"Xyz");
        ht.put(40,"Pqr");

        for(Map.Entry m:ht.entrySet())
        {
            System.out.println(m.getKey()+" "+m.getValue());
        }
    }
}
```

Output



```
10 Abc
20 Lmn
30 Xyz
40 Pqr
```

Difference between HashMap & Hashtable

HashMap	Hashtable
1) HashMap is non synchronized. It is not-thread safe and can't be shared between many threads without proper synchronization code.	Hashtable is synchronized. It is thread-safe and can be shared with many threads.
2) HashMap allows one null key and multiple null values.	Hashtable doesn't allow any null key or value.
3) HashMap is fast.	Hashtable is slow.
4) We can make the HashMap as synchronized by calling this code <code>Map m = Collections.synchronizedMap(hashMap);</code>	Hashtable is internally synchronized and can't be unsynchronized.
5) HashMap is traversed by Iterator.	Hashtable is traversed by Enumerator and Iterator.
6) Iterator in HashMap is fail-fast.	Enumerator in Hashtable is not fail-fast.
7) HashMap inherits AbstractMap class.	Hashtable inherits Dictionary class.

- The **Java Comparable interface**, **java. lang. Comparable** , represents an object which can be compared to other objects.
- For instance, numbers can be compared, strings can be compared using alphabetical comparison etc.
- It provides a single sorting sequence only
- For example, it may be rollno, name, age or anything else.

compareTo(Object obj)

It returns

- **positive integer**, if the current object is greater than the specified object.
- **negative integer**, if the current object is less than the specified object.
- **zero**, if the current object is equal to the specified object.

Example

```
public class Employee implements Comparable<Employee>
{
    int id;
    String name;

    Employee(int id, String name)
    {
        this.id=id;
        this.name=name;
    }
}
```

```
public int compareTo(Employee e)
{
    if(id==e.id)
    {
        return 0;
    }
    else if(id>e.id)
    {
        return 1;
    }
    else
    {
        return -1;
    }
}
```

```
public class sample
{
    public static void main(String[] args)
    {
        ArrayList<Employee> list=new ArrayList<Employee>();
        list.add(new Employee(102,"John"));
        list.add(new Employee(107,"Abc"));
        list.add(new Employee(106,"Raj"));

        Collections.sort(list);

        for(Employee e:list)
        {
            System.out.println(e.id+" "+e.name);
        }
    }
}
```

Output



```
102 John
106 Raj
107 Abc
```

Comparator interface

`compare()` method

- A **comparator interface** is used to order the objects of user-defined classes.
- A **comparator** object is capable of comparing two objects of two different classes.

Example

```
class Student
{
    int rollno;
    String name;

    public Student(int rollno, String name)
    {
        this.rollno = rollno;
        this.name = name;
    }

    public String toString()
    {
        return this.rollno + " " + this.name;
    }
}
```



```
class Sortbyroll implements Comparator<Student>
{
    public int compare(Student a, Student b)
    {
        return a.rollno - b.rollno;
    }
}
```

```
class Sortbyname implements Comparator<Student>
{
    public int compare(Student a, Student b)
    {
        return a.name.compareTo(b.name);
    }
}
```

```
public class sample1 {  
  
    public static void main(String[] args)  
    {  
        ArrayList<Student> ar = new ArrayList<Student>();  
        ar.add(new Student(111, "bbbb"));  
        ar.add(new Student(131, "aaaa"));  
        ar.add(new Student(121, "cccc"));  
  
        System.out.println("Unsorted");  
        for (int i = 0; i < ar.size(); i++)  
            System.out.println(ar.get(i));  
    }  
}
```

```
        Collections.sort(ar, new Sortbyroll());  
  
        System.out.println("\nSorted by rollno");  
        for (int i = 0; i < ar.size(); i++)  
            System.out.println(ar.get(i));  
  
        Collections.sort(ar, new Sortbyname());  
  
        System.out.println("\nSorted by name");  
        for (int i = 0; i < ar.size(); i++)  
            System.out.println(ar.get(i));  
    }  
}
```

Output

111 bbbb

131 aaaa

121 cccc

Sorted by rollno

111 bbbb

121 cccc

131 aaaa

Sorted by name

131 aaaa

111 bbbb

121 cccc



- **Java Generic** methods and **generic** classes enable programmers to specify, with a single method declaration, a set of related methods, or with a single class declaration, a set of related types, respectively.
- **Generics** also provide compile-time type safety that allows programmers to catch invalid types at compile time.

- All generic method declarations have a type parameter section delimited by angle brackets (< >) that precedes the method's return type (< E >).
- **E** is meant to be an Element
- A generic method's body is declared like that of any other method. Note that type parameters can represent only reference types, not primitive types (like int, double and char).

Example

```
import java.util.*;

public class sample
{
    public static < E > void display( E[] arr)
    {
        for(E element : arr)
        {
            System.out.println(element);
        }
    }
}
```

```
public static void main(String args[])
{

    Integer[] intArray = { 1, 2, 3, 4, 5 };
    Double[] doubleArray = { 1.1, 2.2, 3.3, 4.4 };
    Character[] charArray = { 'H', 'E', 'L', 'L', 'O' };

    System.out.println("Integer Array=");
    display(intArray);

    System.out.println("\nDouble Array=");
    display(doubleArray);

    System.out.println("\nCharacter Array=");
    display(charArray);
}
}
```

Output

Integer Array=

1
2
3
4
5

Double Array=

1.1
2.2
3.3
4.4

Character Array=

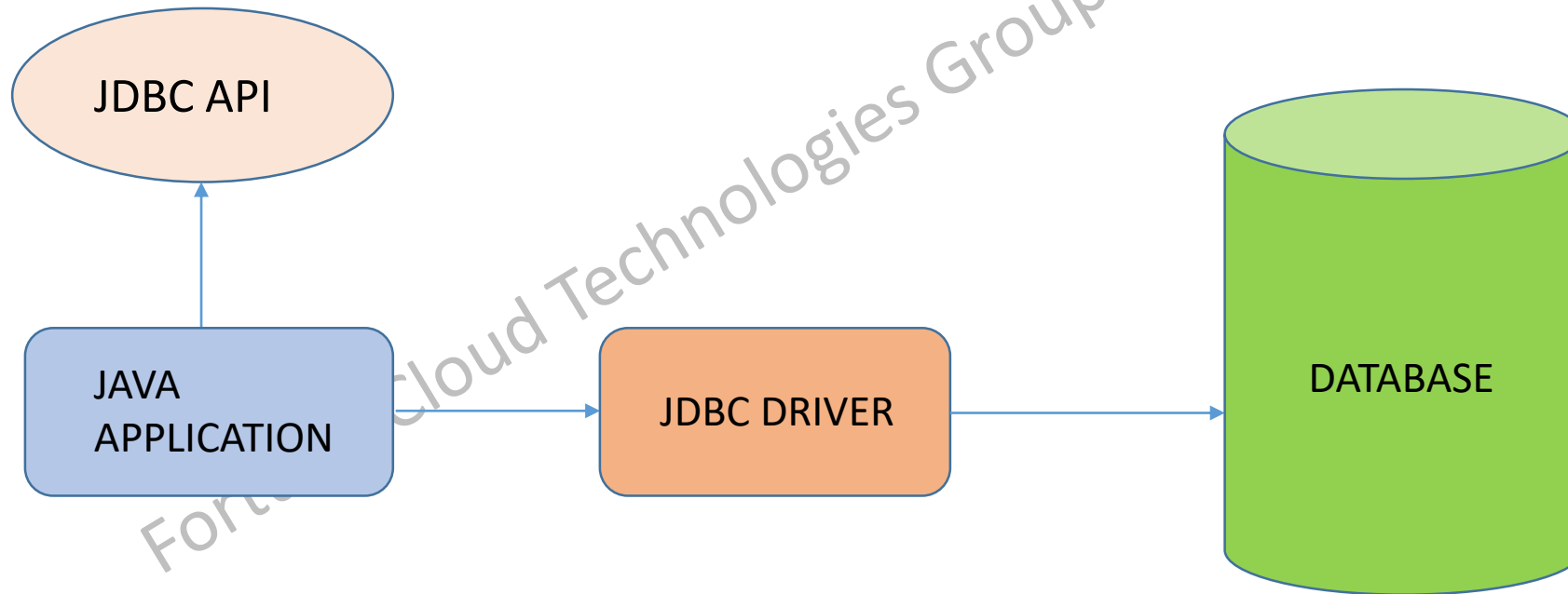
H
E
L
L
O

	Non-generic Collection	Generic Collection
Syntax	<code>ArrayList list = new ArrayList();</code>	<code>ArrayList list = new ArrayList();</code>
Type-safety	Can hold any type of data. Hence not type-safe.	Can hold only the defined type of data. Hence type safe.
Type casting	Individual type casting needs to be done at every retrieval.	No type casting is needed.
Compile-Time Checking	Checked for type safety at runtime.	Checked for type safety at Compile-time.

JDBC

- JDBC stands for **Java Database Connectivity**.
- JDBC is a Java API to connect and **execute the query** with the **database**.
- JDBC API uses JDBC drivers to connect with the database.
- We can use JDBC API to access **tabular data stored** in any relational database.
- By the help of JDBC API, we can **save, update, delete** and **fetch** data from the database
- The **java.sql** package contains classes and interfaces for JDBC API.

JDBC Interface with Application



Database Connectivity Steps

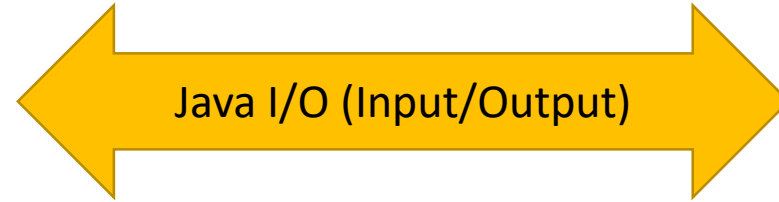


Register the driver class
`Class.forName("com.mysql.jdbc.Driver");`

Create the connection object
`Connection con=DriverManager.getConnection(
"jdbc:mysql://localhost:3308/databasename","username" ,"pass");`

Create the Statement object
`Statement stmt=con.createStatement();`

Execute the query
`ResultSet rs=stmt.executeQuery("select * from emp");`



- **Java IO** is an API that comes with **Java** which is targeted at reading and writing data (input and output).
- For instance, read data from a file or over network, and write to a file or write a response back over the network.
- The **java.io** package contains all the classes required for input and output operations.
- We can perform file handling in **Java** by **Java I/O API**.

FileOutputStream Class

- **FileOutputStream** is an outputstream for writing data/streams of raw bytes to file or storing data to file.
- **FileOutputStream** is a subclass of `OutputStream`. To write primitive values into a file, we use **FileOutputStream** class

Example

```
import java.io.FileOutputStream;

public class sample{
    public static void main(String[] args){
        try
        {
            FileOutputStream fout=new FileOutputStream("D:\\files\\demo.txt");
            String s="I am file output stream";

            byte b[]=s.getBytes();
            fout.write(b);
            fout.close();

            System.out.println("Done");
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```



FileInputStream Class

- **Java FileInputStream class** obtains input bytes from a file. It is used for reading byte-oriented data (streams of raw bytes)

Example- Read Single Character

```
import java.io.FileOutputStream;

public class sample{
    public static void main(String[] args){
        try
        {
            FileInputStream fin=new FileInputStream("D:\\files\\demo.txt");
            int i=fin.read();
            System.out.print((char)i);

            fin.close();
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```


Example- Read All Characters

```
import java.io.FileInputStream;

public class sample{
    public static void main(String[] args){
        try
        {
            FileInputStream fin=new FileInputStream("D:\\files\\demo.txt");
            int i=0;
            while((i=fin.read())!=-1)
            {
                System.out.print((char)i);
            }
            fin.close();
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

BufferedOutputStream Class

- **Java BufferedOutputStream** class is used for buffering an output stream.
- It internally uses buffer to store data. It adds more efficiency than to write data directly into a stream.

Example

```
import java.io.*;

public class sample {
    public static void main(String[] args) {
        try
        {
            FileOutputStream fout=new FileOutputStream(" D:\\files\\demo.txt ");
            BufferedOutputStream bout=new BufferedOutputStream(fout);

            String s="BufferedOutputStream";
            byte b[]=s.getBytes();
            bout.write(b);
            bout.flush();
            bout.close();
            fout.close();
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

BufferedInputStream Class

- A **BufferedInputStream** adds functionality to another input stream-namely, the ability to buffer the input.
- When the **BufferedInputStream** is created, an internal buffer array is created.
- When the bytes from the stream are skipped or read, the internal buffer automatically refilled from the contained input stream
- When a BufferedInputStream is created, an internal buffer array is created.

Example

```
import java.io.*;

public class sample {
    public static void main(String[] args) {
        try
        {
            FileInputStream fin=new FileInputStream(" D:\\files\\demo.txt ");
            BufferedInputStream bin=new BufferedInputStream(fin);

            int i;
            while((i=bin.read())!=-1)
            {
                System.out.print((char)i);
            }
            bin.close();
            fin.close();
        }
        catch(Exception e){
            System.out.println(e);
        }
    }
}
```



FileWriter Class

- **Java FileWriter** class of **java.io** package is used to write data in character form to file.
- **FileWriter** is meant for writing streams of characters.
- For writing streams of raw bytes, consider using a **FileOutputStream**.

Example

```
import java.io.*;

public class sample {
    public static void main(String[] args) {
        try
        {
            FileWriter fw=new FileWriter(" D:\\files\\demo.txt ");
            fw.write("I am FileWriter");
            fw.close();
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

FileReader Class

- **FileReader** makes it possible to read the contents of a file as a stream of characters.
- It works much like the `FileInputStream` except the `FileInputStream` reads bytes, whereas the **FileReader** reads characters.
- The **FileReader** is intended to read text, in other words.

Example

```
import java.io.*;

public class sample {
    public static void main(String[] args) {
        try
        {
            FileReader fr=new FileReader(" D:\\files\\demo.txt ");

            int i;
            while((i=fr.read())!=-1)
            {
                System.out.print((char)i);
            }

            fr.close();    }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

BufferedWriter Class

- BufferedWriter is a sub class of java.io.Writer class.
- BufferedWriter writes text to character output stream, buffering characters so as to provide for the efficient writing of single characters, arrays, and strings.
- Java BufferedWriter class is used to provide buffering for Writer instances. It makes the performance fast.

Example

```
import java.io.*;

public class sample {
    public static void main(String[] args) {
        try
        {
            FileWriter writer = new FileWriter(" D:\\files\\demo.txt ");
            BufferedWriter buff = new BufferedWriter(writer);

            buff.write("I am BufferedWriter");
            buff.close();
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

BufferedReader Class

- Java BufferedReader class is used to read the text from a character-based input stream.
- It can be used to read data line by line
- It makes the performance fast. It inherits Reader class.

Example 1

```
import java.io.*;

public class sample {
    public static void main(String[] args) {
        try
        {
            FileReader fr= new FileReader(" D:\\files\\demo.txt ");
            BufferedReader br=new BufferedReader(fr);

            int i;
            while((i=br.read())!=-1){
                System.out.print((char)i);
            }
            br.close();
            fr.close();
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

Example 2

```
import java.io.*;

public class sample {
    public static void main(String[] args) {
        try
        {
            String name;

            InputStreamReader r=new InputStreamReader(System.in);
            BufferedReader br=new BufferedReader(r);

            System.out.println("What is your name ?");
            name=br.readLine();

            System.out.println("I am "+name);
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```



**Thank
You**